Three-hour teaching workshop

# Agentic AI for Grid Operators

## A worked teaching sequence

Voltage support, restoration and cyber response

This guide explains the ideas behind the native LaTeX slides. It connects physical decisions to numerical tools, workflow stages and the notebook experiments. Read the small example first, then the formal method, then the result.

**How to use it**

Keep the completed-slide PDF beside this guide. The frame numbers below match that PDF. Use the lecture PDF for progressive reveals. Run the complete notebook cells in order; excerpts on slides are reading aids.

All drawings in the teaching sequence use editable LaTeX/TikZ or recorded numerical plots. The introductory examples are labeled illustrative. No AI-generated imagery is used.

# Contents

Before the terminology: one complete decision 2

1 Foundations 4

2 Voltage support 18

3 Break 28

4 Restoration 29

5 Cyber response 37

6 Evaluation and operator decisions 45

## Before the terminology: one complete decision

Suppose an operator has a proposed control setting and wants to know whether it respects a current limit. We first use an intentionally simple model so that every step can be calculated by hand:

$$i = 0.4 + 0.8u, \qquad i \leq 0.8.$$

Here $u$ is the toy control setting and $i$ is the model's current indicator. These numbers teach the workflow; they are not a feeder power-flow equation or a device setting.

| Responsibility | Observable calculation or result |
|---|---|
| Initial information | Proposed $u = 0.75$; current limit 0.8. |
| Simulation | $i = 0.4 + 0.8(0.75) = 1.0$. |
| Verification | $1.0 > 0.8$: reject this proposal. |
| Allowed repair | Reduce the control by 0.25: new $u = 0.50$. |
| New simulation | $i = 0.4 + 0.8(0.50) = 0.80$. |
| New verification | $0.80 \leq 0.80$: the toy current condition passes. |
| Operator record | Identify the exact checked proposal before acknowledgment. |

The graph notebook implements this example with ordinary Python functions inside LangGraph. The repair is a specified rule, not an LLM inference. This makes it possible to test the orchestration separately from the engineering method.

An agentic workflow becomes useful when the next permitted study depends on returned evidence or on an operator request that needs interpretation. The language model can propose a tool call or explain a result. The numerical method still calculates the electrical consequence. A check still decides whether the result meets its stated requirements.

### Five stage names, with concrete jobs

**Awareness** constructs the current problem: measurements, topology, limits and the requested study. **Planning** produces candidate actions using an appropriate method, such as MPC or a restoration MILP. **Simulation** computes the consequence of those actions in a stated model. **Verification** checks the numerical limits, data binding and required scope. The **LLM Optimizer** proposes or selects a permitted refinement, or retains the current choice. Any changed action needs new evidence. Human review follows these stages.

The method changes across cases. Continuous inverter settings suit MPC and continuous-action policies such as SAC. Binary restoration choices suit a mixed-integer formulation. Cyber response starts with measurement estimation and explicit trust assumptions before a constrained corrective action. Giving all these tools an "agent" label does not change their mathematics.

### A diagram-reading key

A box names an operation and its input/output. An arrow identifies what must happen before the next operation can use its result. A branch shows an explicit decision rule. A return arrow means repeat a study after a changed proposal. A limit on repeats prevents an open-ended loop. In the lecture, the same physical drawing stays visible while each calculation or branch appears.

### Preparation and timing

Use Python 3.12 and start Jupyter from the parent tutorial folder. Prepare separate environments from `requirements_graph_lab.txt` and `requirements_linear.txt`. The executed HTML

notebooks provide a reading route when a kernel is unavailable. Install the full OpenDSS requirements before class if you plan to enable that path. Prepare the existing Ollama service and model before enabling L11; the tutorial does not require an in-class model download.

| Minutes | Topic |
|---|---|
| 0–40 | Foundations and G0–G5; L0–L1 |
| 40–80 | Voltage support and L2–L5 |
| 80–90 | Break |
| 90–115 | Restoration and L6–L7 |
| 115–142 | Cyber response and L8–L9 |
| 142–150 | Evaluation, optional L11 and latency |
| 150–180 | Control-room examples, deployment, L12 and synthesis |

The actual archived five-stage responses in L10 are revisited during the case discussions. A saved response is distinct from a fresh numerical calculation and from a new live language-model call.

# 1 Foundations

### Frame 1: An alarm is a question: what can the operator change?

Begin by asking what an operator would need before changing a command. Do not begin with a definition of an agent. The line drawing is a physical motivation, not an electrical model of the Greensboro feeder. Its purpose is to separate a proposed command from the consequence that must be checked. The upcoming arithmetic is deliberately invented and dimensionless so everyone can reproduce it without a simulator. Later, actual feeder tools will replace that arithmetic while the evidence discipline remains. Reveal the operator requirements only after one response from the room. The expected interpretation is that an alarm identifies a problem, but it does not uniquely identify an acceptable action. Next, turn the question into a fully specified calculation.

**Sources.** Teaching schematic; Graph lab G0–G5 declares the toy scope.

### Frame 2: Work the alarm: the proposed command gives too much current

Read each symbol aloud before substituting the numbers. The command u is dimensionless, the calculated current i is a normalized illustrative quantity, and L is its limit in the same normalization. The constants define an invented response law. Multiplication gives 0.60, so adding the offset gives 1.00. The comparison with 0.80 is a separate step from that arithmetic. Ask whether the calculation itself failed; it did not. It correctly identified an inadmissible candidate. Reveal the point on the number line, then the verdict. The expected interpretation is that simulation and verification answer different questions. A converged or correctly executed calculation is not automatically an acceptable result. Next, change the command using an explicit repair rule.

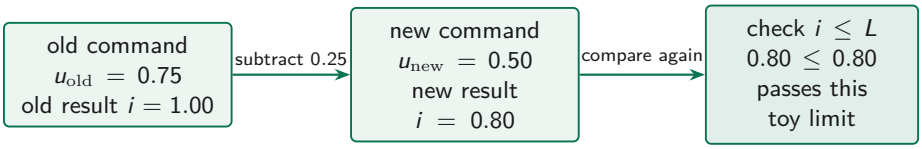
**Sources.** Exact response and comparison used in G2 and G4.

### Frame 3: Repair the command, then recalculate its consequence

Foundations: from an alarm to a checked action

**Repair the command, then recalculate its consequence**

$$u_{\text{new}} = \max(0, u_{\text{old}} - 0.25) = \max(0, 0.75 - 0.25) = 0.50.$$




$u_{\text{old}}$ and $u_{\text{new}}$ are the commands before and after repair. The lower bound of zero prevents a negative command. The old current belongs to the old command. Reusing it would give the wrong decision about the repaired command.

Repair and simulation functions: G2.



Work the subtraction first and ask the class to calculate the new current. The repair produces 0.50, and the response law produces 0.80. Equality passes because the declared rule is less than or equal to the limit. The zero in the maximum is a bound on the command, not an extra simulation result.

The important process lesson is that changing the command invalidates the previous current as evidence for that new command. A record may preserve the old value as history, but a new verification must use the new calculation. Reveal the last box only after the audience predicts the result. The expected interpretation is a complete candidate, calculation, and comparison sequence. Next, name those responsibilities without yet introducing software terminology.

**Sources.** Repair and simulation functions: G2.

### Frame 4: The operator's calculation already has a workflow

The four boxes are now meaningful because each contains a number the audience has already calculated. Read the arrows as requirements, not as electrical lines. The current must exist before the comparison can be performed, and the command must exist before its consequence can be calculated. These dependencies explain why a workflow is useful without relying on a vague automation claim. Ask what would be lost if only the final word pass were stored. The audience should identify the command, limit, or model assumptions that make that word interpretable. A later language model can help select permitted work, but it does not eliminate these dependencies. The expected interpretation is that the workflow organizes familiar engineering responsibilities. Next, put the results into a concrete review record.

**Sources.** Tutorial decomposition; executable functions in G1–G3.

### Frame 5: A reviewable recommendation is more than a pass label

Read the recommendation record as something an operator can challenge. The command says what is proposed, the consequence says what was calculated, and the scope says how far the calculation applies. The identity field ties a response to this recommendation rather than to a vaguely remembered previous one. Ask whether a changed command should inherit the previous acknowledgment. The expected answer is no, even if the numerical difference is small. In the graph lab, the response is scripted inside the notebook and represents a simulated readback. It demonstrates state handling rather than authentication or dispatch. The later engineering packet uses the same distinction between evidence and acknowledgment. Next, place this small example within the full teaching route and its three physical cases.

**Predict before revealing** Would the same acknowledgment apply after changing $u$?

**Explanation** No. A changed recommendation needs its own evidence and matching readback.

**Sources.** Review payload and identity test: G5; engineering handoff: L12.

## Frame 6: Three hours: understand a process, then challenge it in code

Foundations: from an alarm to a checked action

### Three hours: understand a process, then challenge it in code

| Minutes | Question we will answer | Hands-on checkpoint |
|---|---|---|
| 0–40 | How do state, tools, and decisions connect? | G0–G5; L0–L1 |
| 40–80 | Which inverter command meets the limits? | L2–L5; Llama evidence |
| 80–90 | Break | Keep the kernels available |
| 90–115 | Which permitted restoration choice serves priorities? | L6–L7 |
| 115–142 | Which measurements can support a response? | L8–L9 |
| 142–180 | What does the evidence justify for the operator? | L10–L12; capstone |

Every exercise follows **predict, run, explain**. A fresh numerical result, an archived actual Llama reply, and a new Llama call have different labels.

Executable teaching route: graph notebook and engineering notebook.

**Next:** Which repeated parts of our calculation are worth automating? 6/138

Use the timetable to set a practical pace. Installation is a prerequisite; the class should not begin installing a large language model during the voltage exercise. Participants need the graph notebook and the classroom notebook, with the working directory set as documented. The first uses transparent arithmetic to expose execution mechanics. The second connects numerical methods, stored learned weights, and explicitly labeled language-model evidence. Preserve the difference between an archived reply and a newly generated reply when discussing latency or reproducibility. The course includes a ten-minute break and time to interpret outputs rather than continuously advance slides. The expected interpretation is a sequence of connected experiments, not a slide lecture followed by unrelated code. Next, examine which repeated responsibilities automation can make explicit.

**Sources.** Executable teaching route: graph notebook and engineering notebook.

## Frame 7: Automate the repeated work; keep its evidence visible

Begin with work the audience recognizes: apply the response law again, compare with a limit again, and record what happened. A fixed supervisor can automate all of it. This is important because the tutorial should not assign ordinary software benefits exclusively to language models. The agentic contribution must be identified and tested, such as selecting useful follow-up work from a bounded catalog or helping assemble an interpretable response. These contributions can be helpful while leaving the numerical action unchanged, and they can introduce latency. The graph baseline later makes this distinction concrete by matching all three toy outcomes. The expected interpretation is a testable motivation for the architecture rather than an assumption of superiority. Next, name the components by the jobs they actually perform.

**Sources.** Matched evaluation: PowerAgentBench-SS, 2026 preprint; tutorial baseline: G4.

## Frame 8: Three names for three jobs we have already seen

The diagram deliberately places the orchestrator above the interaction instead of after the tool in a false linear pipeline. Point to the request and returned evidence. The tool performs a specific

calculation. The agent role chooses an allowed operation using the current context. The orchestrator makes the sequence and state management explicit. These responsibilities can be implemented in several ways, and their names do not imply independent intelligence or independent errors. The toy's choice policy is ordinary Python, so it does not become a language-model experiment when drawn as a graph. Actual Llama calls appear in the engineering records later. The expected interpretation is a clear division of responsibility. Next, show a bounded language-model decision and the numerical work that must follow it.

**Sources.** Role-based systems: AutoGen; tutorial separation: workflow implementation.

### Frame 9: Llama can choose follow-up work; the tool supplies the numbers

Make the evidence label explicit before discussing the example. The prompt and two-operation catalog illustrate a design; they are not an actual model response and are not executed by the toy. The later engineering examples use a different declared operation catalog and contain real archived calls. The language model can help interpret a request and select bounded follow-up work. It cannot make the repaired command acceptable merely by describing it favorably. Numerical evaluation and an unchanged acceptance rule remain necessary. Public explanations can be compared with returned evidence without requesting hidden private reasoning. The expected interpretation is a constrained decision interface with inspectable inputs and outputs. Next, connect that interface to the proposal's complete sequence of responsibilities.

**Sources.** Observe–act feedback: ReAct; actual replies: L3, L10, L11.

### Frame 10: All five proposed stages have a concrete return value

Carry one candidate through the five boxes and name the returned artifact at every stage. Awareness establishes the request and model scope. Planning returns a candidate. Simulation returns modeled consequences. Verification returns numerical margins and a verdict. The LLM Optimizer chooses permitted follow-up work, and a changed candidate returns to simulation and verification. Human review occurs after this evidence is assembled. The toy variables illustrate the handoff but do not imply that the toy itself contains five LLM calls. The engineering archive later exposes all five actual stage records, including deterministic events. The expected interpretation is a complete process with an explicit return loop, not five independent votes. Next, select algorithms according to the underlying mathematical problem rather than the role name.

**Sources.** All five stages and recorded events: executed tutorial trace.

### Frame 11: The workflow reuses numerical methods; it does not replace them

The method should match the decision variables and evidence available in the case. Voltage control uses continuous inverter commands, so MPC and a learned continuous-action actor are natural examples. Restoration includes discrete service decisions and therefore needs a different formulation. The cyber example begins with measurement evidence and state estimation before a constrained response is considered. The framework coordinates these operations and records their results; it does not make one algorithm universally appropriate. Introduce model-based and model-free at this general level before the detailed equations later. A model-free actor can be trained using simulated experience, which is not a contradiction. The expected interpretation is methods inside the workflow. Next, inspect the common state record that prevents information from being lost between these methods.

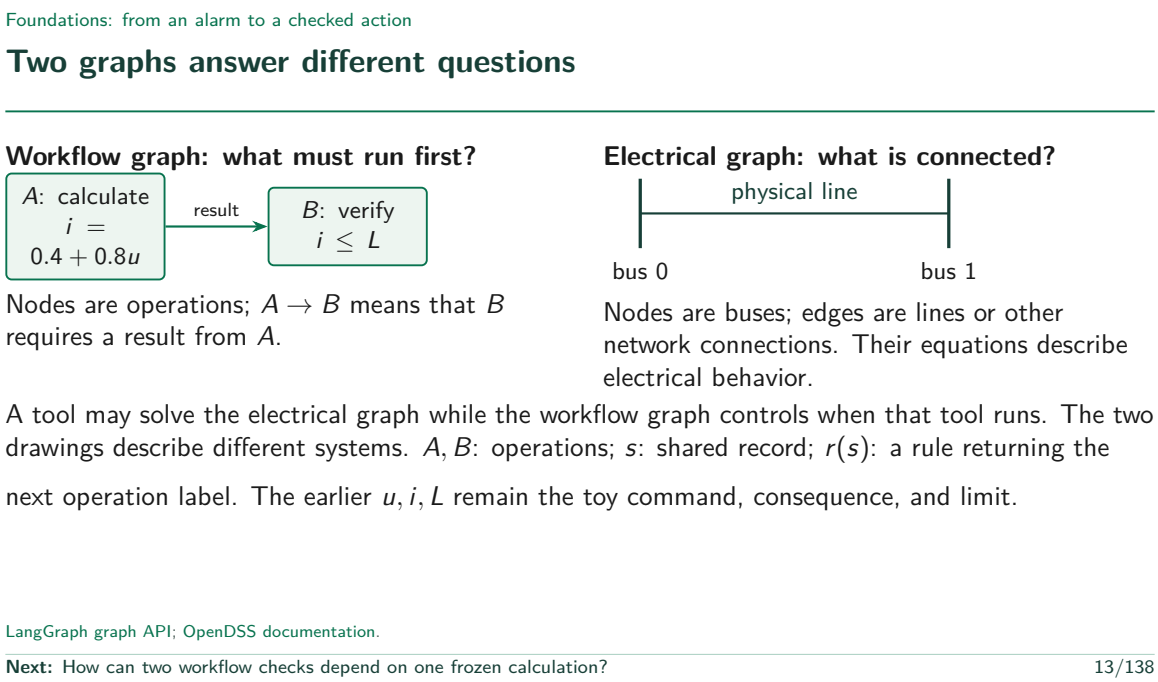
**Sources.** Methods: classroom tools; MPC; SAC.

### Frame 12: State is the record that keeps this decision coherent

Use the paired tables as a before-and-after comparison. The command and calculated current change, the verdict changes, and the attempt counter increments. The limit does not change. This makes the separation between changing an action and changing the acceptance criterion concrete. State is more useful than an unstructured conversation for this purpose because each relevant field can be located and checked. The symbol s with a subscript t denotes the entire record before a computational step, not the electrical system state alone. Nodes return updates rather than rewriting unrelated fields. Ask which old value may be preserved in history but must not remain the current result. The expected answer is 1.00. Next, use a graph to display which operations need which record fields.

**Sources.** Exact fields and updates: G1–G2.

### Frame 13: Two graphs answer different questions




Read the left and right headings before following either set of edges. The workflow graph describes computations: calculate a result, then use it in verification. The electrical graph describes physical connectivity between buses. A line in the right-hand picture does not specify a software execution order. A power-flow tool can solve equations on that network while the workflow controls when to call the tool and what to do with its output. Ask which drawing would change if an electrical switch opened, and which would change if a review step were added. This distinction will return in restoration, where topology is part of the physical decision. The expected interpretation is two graph meanings with separate node and edge definitions. Next, branch two computations from one frozen input.

**Sources.** LangGraph graph API; OpenDSS documentation.

### Frame 14: A DAG can branch two checks and join their evidence

The example now branches only after the input and calculated current have been frozen. One operation checks the current limit, and another checks the declared command range. Both use the same candidate. The final operation combines their results and must wait for both. Define the graph, node set, and edge set directly under the drawing so no symbol is unexplained. A directed

acyclic graph has no path that follows arrows back to its starting node. These conceptual checks might be independent, but the picture does not claim that the engineering adapters execute concurrently. It also differs from the cyclic graph in the lab. The expected interpretation is a prerequisite structure with a genuine join. Next, ask the audience to choose valid schedules.

**Sources.** DAG ordering: NetworkX; illustrative checks use the declared toy variables.

### Frame 15: A valid schedule puts both checks before the join

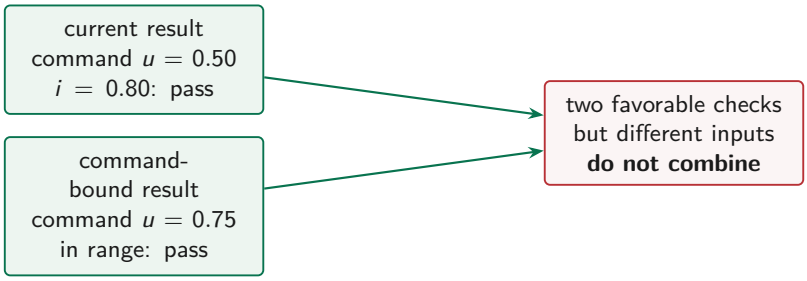
Pause before revealing the validity column. Ask participants to name the dependency violated by the third sequence. The final join requires the command-bound result, so it cannot run before C. The other two sequences respect every arrow. This is the practical meaning of a topological order. The braces sometimes used to group parallel tasks are not needed to answer this question; valid sequential schedules already explain the dependency. If a participant proposes parallel execution, acknowledge that the graph permits independent work but that real tools may share state or resources. The tutorial does not execute OpenDSS concurrently. The expected interpretation is ordering rather than speed. Next, show why correct order is insufficient when two results refer to different frozen inputs.

**Sources.** Topological ordering: NetworkX.

### Frame 16: The join must match the candidate, not merely collect two ticks

Foundations: from an alarm to a checked action

**The join must match the candidate, not merely collect two ticks**




Both calculations can be individually correct. Together they do not establish the stated checks for one common candidate. Bind each result to the request revision and command. In L1, changing either makes the old binding fail.

Exact request/candidate binding exercise: L1.

**Next:** What happens when a failed check deliberately sends us backward? 16/138

This deliberately mismatched example shows why a collection of favorable results is not automatically a complete verdict. One result belongs to command 0.50 and the other to command 0.75. Both can be mathematically correct while being inappropriate to combine as evidence for one candidate. The issue is identity, not arithmetic. Ask the class to identify the missing common condition before revealing the binding rule. The classroom implementation later binds request, candidate, and result with hashes and rejects a mismatch. That establishes consistency of recorded data, not source authenticity. The expected interpretation is that a join checks identity as well as completeness. Next, add a repair edge and distinguish the resulting cyclic workflow from the acyclic dependency example.

**Sources.** Exact request/candidate binding exercise: L1.

### Frame 17: A repair loop is a cycle, so give it a stopping rule

Trace the successful route first and then reveal the backward repair edge. The graph now contains a cycle, so calling it a DAG would be incorrect. A particular completed run can still be drawn as a sequence of visits. The counter k records completed repairs and K is the maximum permitted count. The policy repairs only while k is smaller than K; otherwise a failed verification produces hold. The finite count bounds attempts if each operation returns, but it is separate from a timeout for a stalled solver. Ask whether stopping without a feasible action means the software failed. It may be the correct policy result. The expected interpretation is bounded execution with an explicit unsuccessful outcome. Next, write the routing conditions precisely.

**Sources.** Exact bounded policy: G2–G4; LangGraph.

### Frame 18: Route from the verdict and the remaining budget

Reveal the mutually exclusive conditions in order. The first condition checks whether the latest calculation passed. If it did not, the second decision is whether a repair is still permitted. The route returns a label, and the graph maps that label to a node. This policy is deliberately deterministic so that participants can reason about every execution path. A more flexible language-selected operation could be inserted elsewhere without allowing it to bypass these acceptance conditions. Ask what would happen if the counter were equal to the budget: the route must hold. The numerical verification rule and the orchestration policy remain separate functions.

**Predict before revealing** A failed check has $k = 1$ and $K = 1$. Which route is permitted?

**Explanation** `hold`. The budget is exhausted; equality does not permit another repair.

**Sources.** The implemented route is four Python lines in G2.

### Frame 19: A state schema names the facts each node may need

Read the displayed record as a laboratory notebook with named fields. The command identifies what was proposed, the evidence fields identify what was calculated, and the control-flow fields identify what should happen next. The merge expression is a conceptual description of applying returned updates. Delta is not a numerical difference between two entire records. The G notebook uses TypedDict to describe expected keys and types, while its initial-state function explicitly validates numerical inputs. TypedDict itself does not perform runtime value checking. A chat transcript alone is not a substitute for this engineering record because important values and units must be easy to locate and compare.

**Sources.** Python TypedDict; validated inputs in graph-lab helper.

### Frame 20: A reducer states whether new information replaces or appends

Use the animation to replace the scalar value while visibly extending the event list. The current field describes the latest simulated command, so retaining the old value there would be misleading. The trace field describes history, so discarding earlier events would make the repair difficult to inspect. The example's plus symbol means list concatenation, not arithmetic averaging or consensus. In the code, Annotated attaches the addition function to the trace field. An empty appended list does not erase existing events. If several branches write a field, the application needs a merge rule appropriate to that field and its ordering semantics.

**Sources.** List reducer in G1; LangGraph reducers.

### Frame 21: Three starting states test three distinct paths

Give the audience time to calculate the three first results before revealing the outcomes. The first starts within its limit and needs no repair. The second is the worked example: one subtraction produces a command of 0.50 and a current of 0.80. The third starts at 1.12, repairs to 0.92, and still exceeds its smaller limit of 0.50. Its one permitted repair has been used, so a hold is correct. We have not run code yet; these are predictions that the later assertions will test. Ask whether a more generous budget changes the third outcome. The expected interpretation is that success, successful repair, and exhausted repair are different paths worth exercising. Next, explain the runtime that connects the functions.

**Sources.** These are the exact three cases executed in G4.

### Frame 22: LangGraph runs the stateful process we have already specified

Introduce the package only after the audience understands the process it will execute. LangGraph does not invent the response law or choose the acceptance rule in this example. Those remain ordinary Python functions. The graph definition supplies a state schema, nodes, edges, and a conditional route, while the runtime supports execution and later pause mechanisms. This separation makes it easier to compare the graph with a direct-Python implementation using the same functions. The G notebook is an actual LangGraph exercise, but the larger engineering notebooks use their existing custom orchestration. The expected interpretation is a precise software boundary rather than a package name attached to everything. Next, run setup and inspect the installed versions and the scope statement.

**Sources.** LangGraph overview; executed G notebook.

### Frame 23: G0: import the runtime and record what this experiment is

Run the complete G0 cell, even though the slide displays only its relevant lines. Confirm that the version dictionary appears before proceeding. The captured run used LangGraph 1.2.15, checkpoint 4.3.0, and langchain-core 1.6.12; a participant's environment may differ. The cell disables the external tracing flags used by this exercise and declares that no language-model or electrical-model validation occurs. After dependencies are installed, the toy needs no Ollama service. If imports fail, use the supplied graph-lab requirements and return to the documented setup rather than changing the experiment during class. This practical boundary keeps the graph lesson reproducible. The expected interpretation is to identify the input, returned evidence, and next permitted step before discussing the final status.

**Notebook excerpt.** Run the complete cell in context.

```
os.environ["LANGSMITH_TRACING"] = "false"
os.environ["LANGCHAIN_TRACING_V2"] = "false"
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt, Command
from src.graph_workflow_lab import initial_state, recommendation_id, summary
versions = {name: version(name) for name in ("langgraph", "langgraph-checkpoint", "langchain-core"
    )}
print(versions)
print("Rule-driven toy graph. No LLM, network call or electrical-model validation.")
```

**Sources.** Exact notebook lines 5–13: G0; LangGraph.

### Frame 24: G1: the schema is our before-and-after table in Python

Highlight the control fields first, then the verdict fields, then the trace annotation. Run the complete G1 cell so the audience also sees the initial validated state and the list-addition example below this excerpt. TypedDict states the intended record structure; initial_state performs the example's explicit input validation. These two mechanisms should not be confused. The trace is a sequence of recorded events, not an internal chain of thought and not a cryptographically authenticated audit log. Ask participants to predict the number of trace entries before revealing the answer. This is the same replace-versus-append idea they saw conceptually. The expected interpretation is to identify the input, returned evidence, and next permitted step before discussing the final status.

**Notebook excerpt.** Run the complete cell in context.

```python
class TrialState(TypedDict):
    command: float
    limit: float
    attempts: int
    max_repairs: int
    current: float
    passed: bool
    status: str
    acknowledged: bool
    trace: Annotated[list[str], add]
```

**Sources.** Exact state schema: G1; TypedDict.

### Frame 25: G2: simulation returns a number; verification returns a verdict

Read the first function as an ordinary Python calculation. It returns current and a trace update rather than reconstructing the entire state record. The next function compares that current with the limit using a small numerical tolerance. The tolerance addresses floating-point comparison in this toy; it is not an engineering margin. With command 0.75, the result is 1.00 and the limit is 0.80, so the verdict is false. That outcome is expected rather than a program error. Ask which returned field the router will inspect. Run the entire G2 cell, including the repair and terminal-node definitions that follow this excerpt.

**Notebook excerpt.** Run the complete cell in context.

```python
def simulate(s):
    value = 0.4 + 0.8 * s["command"]
    return {"current": value, "trace": [f"simulate: {value:.3f}"]}

def verify(s):
    ok = s["current"] <= s["limit"] + 1e-12
    return {"passed": ok, "trace": [f"verify: {ok}"]}
```

**Sources.** Exact functions: G2, lines 1–7.

### Frame 26: G2: repair changes the command; routing checks the budget

Read the two functions as separate responsibilities. Repair computes a changed command and records that an attempt has been used. Route reads the most recent verdict and decides which node may execute next. Neither function relaxes the current limit. With the worked starting command, one repair changes 0.75 to 0.50 and increments the counter from zero to one. The new command still requires simulation and verification before it can report ready. Ask what route is returned after a failed check with one attempt used and a maximum of one. The answer is hold. The expected interpretation is that the stopping policy is visible in four lines, rather than hidden in a prompt. Next, connect these functions using the graph's actual edges.

**Notebook excerpt.** Run the complete cell in context.

```
def repair(s):
    return {"command": max(0., s["command"] - 0.25),
            "attempts": s["attempts"] + 1, "trace": ["repair: -0.25"]}
```

**Notebook excerpt.** Run the complete cell in context.

```
def route(s):
    if s["passed"]:
        return "report"
    return "repair" if s["attempts"] < s["max_repairs"] else "hold"
```

**Sources.** Exact repair and route: G2, lines 9–11 and 19–22.

### Frame 27: G3: the backward edge makes every repair face a fresh calculation

Run G3 and confirm the compiled node list. Highlight the fixed edge from simulate to verify, the conditional edge that calls route, and the edge returning repair to simulate. The mapping translates a returned label into a permitted successor. The report and hold nodes terminate this version of the execution. Compilation checks structural requirements of the graph; it is not a proof that the chosen response law describes a feeder or that the repair rule finds an optimum. This keeps the software lesson separate from the later numerical science. Ask a participant to point to the line that makes the graph cyclic.

**Notebook excerpt.** Run the complete cell in context.

```
    builder.add_edge(START, "simulate")
    builder.add_edge("simulate", "verify")
    builder.add_conditional_edges("verify", route,
        {"report": "report", "repair": "repair", "hold": "hold"})
    builder.add_edge("repair", "simulate")
    builder.add_edge("report", END)
    builder.add_edge("hold", END)
    return builder.compile(checkpointer=checkpointer)
```

**Sources.** Exact graph wiring: G3; LangGraph construction.

### Frame 28: G4: predict, run, then explain one complete trace

Give participants time to predict before running. The first case passes immediately. The second begins at current 1.00, reduces the command from 0.75 to 0.50, and then reaches current 0.80. The third begins at command 0.90, repairs to 0.65, and still exceeds its 0.50 limit with current 0.92. Its single repair is exhausted, so hold is correct. Run G4 unchanged because its assertions describe these original three cases. For a new limit or repair budget, invoke the graph in a separate scratch cell. A useful extension gives the third case four repairs: it reaches command zero and current 0.40. The expected interpretation is to identify the input, returned evidence, and next permitted step before discussing the final status.

**Predict before revealing** What if the exhausted case gets four repairs instead of one?

**Explanation** It reaches $u = 0$, hence $i = 0.40 \leq 0.50$, and can report ready.

**Sources.** Exact outputs, trace assertion, and cases: G4.

### Frame 29: G4: hold the functions fixed and compare the supervisor

Make the null numerical result a central teaching outcome. Both implementations receive the same initial state and the same node functions. The comparison asserts equality of the summary, including the command, status, and trace. Therefore the graph should not be credited with finding a better action. Its value here is making execution structure and later checkpoint mechanisms explicit. The notebook prints elapsed times from one local execution, but those values are not a statistically controlled performance benchmark. Differences at this scale depend on runtime and measurement overhead. Ask which separate experiment would support an inspectability or recovery claim. The expected interpretation is to identify the input, returned evidence, and next permitted step before discussing the final status.

**Notebook excerpt.** Run the complete cell in context.

```
from src.graph_workflow_lab import fixed_baseline
for name, args in cases.items():
    baseline = fixed_baseline(initial_state(*args), simulate, verify, repair, route, ready, hold)
    assert summary(baseline) == summary(results[name])
print("Direct Python baseline matches every command, outcome and trace: 3/3.")
```

**Sources.** Executable baseline equality assertion: G4, lines 15–19.

### Frame 30: A checkpoint preserves the record and the next unfinished step

Use a bookmark analogy, then make it precise with the stored record and pending node. In the G lab, a thread identifier associates the checkpoint with a particular execution. That identifier is not an operator credential. The example uses InMemorySaver, so the state remains available in the running process but does not provide durable recovery after the kernel disappears. This scope is sufficient for observing a pause and resume during class. Participants should inspect the snapshot before discussing its final status; otherwise a whole-cell execution can make the pause easy to miss. The upcoming exercise prints the pending node and acknowledgment flag.

**Sources.** LangGraph persistence; actual checkpoint inspection: G5.

### Frame 31: G5: interrupt with the recommendation that needs a response

Distinguish a visible recommendation from an acknowledged recommendation. The toy sends an interrupt payload that includes a recommendation identifier, command, calculated value, and declared scope. Its scripted reply must approve the same identifier. A wrong identifier or a declined response remains distinguishable from approval. This demonstrates a record-consistency mechanism, not operator authentication or equipment dispatch. On resumption, the interrupted node can begin again, so uncontrolled side effects before the interrupt would be a design problem. The toy only records state changes. Ask participants what the operator should see if the candidate changes while review is pending. The answer motivates fresh evidence and a new identity.

**Notebook excerpt.** Run the complete cell in context.

```
def human_review(s):
    token = recommendation_id(s)
    reply = interrupt({"recommendation_id": token, "command": s["command"],
                       "toy_current": s["current"], "scope": "invented toy equation only"})
    accepted = (isinstance(reply, dict) and reply.get("approve") is True
                and reply.get("recommendation_id") == token)
    return {"acknowledged": accepted,
            "status": "TOY_ACKNOWLEDGED" if accepted else "TOY_HOLD",
            "trace": [f"human review: {accepted}"]}
```

**Sources.** Exact review function: G5, lines 1–9; interrupt semantics.

### Frame 32: G5: pause and inspect before resuming

The full G5 cell both pauses and supplies a scripted response, so explain how to observe the intermediate state. For a live teaching pause, execute lines 1 through 17 first, retaining the function definition above this displayed excerpt. Inspect the printed pending node, which is report, and the false acknowledgment flag. The interrupt payload identifies the command and the invented model scope. In the captured pause, the stored status remains TOY_NEW; it should not be mislabeled TOY_READY merely because the numerical gate passed. The checkpoint exists in this kernel's memory. Participants can now see that a verified numerical result and an acknowledged recommendation are different states.

**Notebook excerpt.** Run the complete cell in context.

```
review_app = build_graph(human_review, InMemorySaver())
config = {"configurable": {"thread_id": "student-" + uuid.uuid4().hex}, "recursion_limit": 40}
pause = review_app.invoke(initial_state(0.75, 0.80, 2), config)
snapshot = review_app.get_state(config)
print("Pending node:", snapshot.next, "| acknowledged:", snapshot.values["acknowledged"])
print("Interrupt payload:", pause["__interrupt__"][0].value)
assert snapshot.next == ("report",) and not snapshot.values["acknowledged"]
```

**Sources.** Exact pause and snapshot: G5, lines 11–17.

### Frame 33: G5: resume the same execution with a matching readback

Execute the remaining lines after the pause, or inspect both printed stages if participants ran the full cell. The response is explicitly simulated and generated by the notebook, not provided by an authenticated operator. Its identifier must match the recommendation constructed from the saved state. The resumed graph records acknowledgment and has no pending node. To discuss rejection, point to the human_review function: it requires an approval value of true and the correct identifier; otherwise it returns hold. Avoid resuming an already completed execution for this experiment; create a fresh review run before trying a different response. No equipment endpoint is present.

**Notebook excerpt.** Run the complete cell in context.

```
response = {"approve": True, "recommendation_id": recommendation_id(snapshot.values)}
resumed = review_app.invoke(Command(resume=response), config)
print("After simulated readback:", summary(resumed))
assert resumed["acknowledged"] and review_app.get_state(config).next == ()
```

**Sources.** Exact resume and final assertion: G5; LangGraph interrupts.

### Frame 34: L1: a changed request must not reuse old evidence

Allow four minutes for setup, prediction, execution, and explanation. The initial request uses load scale 1.00, rating fraction 0.80, and revision one. L1 changes the load to 1.10 and revision to two, then separately tests a changed command. The expected binding results are true, false, and false. The payload is deliberately marked example_only; this is an evidence-identity experiment, not a fresh feeder solve. A matching hash establishes consistency among these recorded inputs and the bound result. It does not establish source authenticity, correct intent, or physical feasibility. Fixed supervisors can use the same guard, so the test demonstrates an implementable workflow property without claiming an exclusive LLM benefit.

**Notebook excerpt.** Run the complete cell in context.

```
request = lab.validate_request(dict(task='voltage', backend='lindistflow3',
    load_scale=1.0, rating_fraction=0.80, revision=1))
candidate = {'control': [0.25, 0.0]}
evidence = lab.bind_evidence(request, candidate, {'example_only': True, 'passed': True})
changed_request = {**request, 'load_scale': 1.10, 'revision': 2}
```

**Sources.** Exact inputs and expected outputs: L1; binding implementation.

### Frame 35: Schema, binding, and engineering checks answer different questions

Keep this slide concrete rather than reading a compliance checklist. Give the current example verbally and ask for the verdict before revealing it. A converged solver result can fail a physical operating condition. A correctly shaped result can be stale. A matching hash can identify consistent data without proving who produced it. These checks are complementary because they answer different questions. Our later lab uses an explicitly illustrative payload to isolate the binding mechanism; participants must not mistake its passed field for a real feeder calculation. The final operator packet adds another distinction between evidence scope and the requested scope.

**Predict before revealing** A solver converges at 112 A; the declared limit is 100 A. Two LLMs favor the candidate. Accept it?

**Explanation** No. The current check fails by 12 A. A vote cannot raise the declared limit.

**Sources.** Tutorial evidence and limit checks: classroom tools; case adapters.

### Frame 36: MCP connects tools; MPC computes a control plan

Spell out both acronyms and contrast their purposes using the two drawings. MCP is a communication interface for tools. MPC is a numerical control method that predicts behavior and optimizes a sequence of actions. The similarity of their letters does not imply a mathematical connection. A solver implementing MPC can be exposed as a tool through an MCP server, but the interface does not replace the objective, constraints, or verification. Our classroom notebook calls Python functions directly and does not claim to demonstrate an MCP server. Ask which component would compute a control sequence and which would describe how to call that component. The expected interpretation is a clean separation between interoperability and control. Next, compare optimization-based planning with a learned continuous-action policy.

**Sources.** MCP tools specification; Rawlings et al., MPC.

### Frame 37: MPC plans with a model; SAC supplies a learned continuous action

Define the two broad approaches before naming the particular algorithms. Model-based control uses predicted behavior in an optimization problem; the displayed expression omits constraints for clarity, and the voltage lesson supplies them explicitly. Model-free learning trains an action rule through experience and rewards. It may obtain that experience from a simulator, so model-free does not mean no simulator was used. MPC plans over a horizon and applies its first action. SAC supplies continuous control from saved learned weights in class. PPO is a relevant continuous-action alternative but is not trained in this tutorial. Training cost and inference cost are separate. Both proposed commands must be simulated and checked under the same declared limits.

**Sources.** MPC; SAC; PPO.

### Frame 38: Prior frameworks contribute mechanisms we can teach and test

After the concrete examples, the audience can now connect literature mechanisms to work they understand. PowerAgent organizes domain capabilities through interfaces and workflows. PowerDAG uses retrieved exemplars and a prerequisite supervisor whose inspected version covers selected rules and allows a repeated call after a one-shot advisory. Grid-Agent separates roles and evaluates changes with validation and rollback. X-GridAgent separates planning, coordination, and action. The point is to identify design ideas rather than compare incompatible success percentages. These works are close prior art, so the tutorial should not claim to invent role decomposition or numerical checking. The expected interpretation is a mechanism-focused literature map. Next, use a small tick matrix while preserving the qualifications a tick alone could hide.

**Sources.** PowerAgent; PowerDAG; Grid-Agent v3; X-GridAgent.

### Frame 39: A capability tick describes documented scope, not superiority

Define the marks before interpreting the rows. A tick records a documented capability in the inspected source; a triangle preserves a qualification; NR means not reported in that source rather than proved absent. PowerAgent capabilities vary by workflow. PowerDAG covers selected dependencies with a one-shot advisory. Grid-Agent oversight is proposed for deployment. X-GridAgent includes solver checks and human prompt refinement. PowerAgentBench-SS supplies an evaluation protocol with hidden revalidation under a DC scope. The tutorial's review tick refers to local advisory readback, not authenticated field authorization. None of these rows forms a shared performance benchmark. Ask what would make a valid benefit claim: common scenarios, tools, budgets, outcome definitions, and timing boundaries.

**Sources.** PowerAgent; PowerDAG; Grid-Agent v3; X-GridAgent; PowerAgentBench-SS.

### Frame 40: Compare benefit in three columns, with the same experiment

Ask the audience to place a claim in one of the three columns before accepting it. A smaller objective is a numerical claim. Rejection of a stale result is a workflow claim. Faster completion is a timing claim. One does not establish the others. The toy's matching commands demonstrate that graph execution can preserve a conventional policy, but do not demonstrate better control. Later comparisons must disclose common scenarios, candidate budgets, model scope, and the boundary of measured time. Training a controller and running inference are different costs, as are a saved language-model reply and a new call. The expected interpretation is a disciplined way to assess useful, null, or worse outcomes. Next, replace the invented response law with the two declared electrical models.

**Sources.** Tutorial matched baseline: G4; evaluation design: PowerAgentBench-SS.

### Frame 41: Keep the process; replace the toy law with electrical models

Recap the transferable mechanisms in one sentence: state records the case, tools return evidence, routes handle success and repair, and review records a response to a particular recommendation. The G lab established those mechanisms but did not establish grid-control performance. The engineering notebook now switches to actual numerical models and recorded model responses. Keep the comparison fair by fixing the scenario, permitted controls, declared constraints, objective, and candidate budget wherever a paired comparison is claimed. Report changes in model scope explicitly rather than treating them as unexplained error. The next section introduces the electrical meaning of active and reactive control before displaying the optimization.

**Sources.** Voltage experiments and exact outputs: L2–L5; response traces: five-stage archive.

# 2 Voltage support

### Frame 42: The operator's question: how much PV can this feeder accept?

The voltage chapter uses one SMART-DS synthetic Greensboro feeder, identified as uhs20_1247–udt8894. It is not a model of the whole city or a live utility. Ask learners which information they still need before advising an operator: device availability, thermal ratings, voltage bounds, model coverage and permitted actions. The map establishes geographic context; it does not establish operational validity. Our numerical experiment uses a fleet command with two continuous variables. All recommendations remain advisory simulation outputs. This chapter moves from a familiar control problem to the reason for explicit agent handoffs: a recommendation should carry the evidence and assumptions that justify it.

**Predict before revealing** Which could limit additional PV: voltage, current, or inverter capability?

**Explanation** Any of the three. We must inspect their margins together.

**Sources.** SMART-DS synthetic distribution models; Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

### Frame 43: See the electrical path before choosing an algorithm

Begin at the source and follow the physical conductor to the load and inverter. The PV array supplies DC power; its inverter provides a controlled AC interface. The feeder impedance makes local voltage depend on both active and reactive flow. Under sufficiently high PV production, net active power can flow toward the source, which can raise downstream voltage. Reactive support is not free: it also changes current and consumes inverter capability. The picture is an explanatory single-branch abstraction, not the topology used by the full numerical model. Ask learners to point to the location of each measured quantity. This establishes a physical reference for the next sign-convention and arithmetic examples.

**Sources.** Gan & Low: three-phase distribution optimization (2014)

### Frame 44: Keep net demand and inverter injection signs distinct

State both sign conventions explicitly. The distribution line sees net demand: positive P and Q mean consumption downstream. The inverter uses an injection convention, so positive reactive injection reduces net reactive demand, while absorption means a negative inverter reactive command and increases net Q. For a conventional line with positive resistance and reactance, increasing net reactive demand increases the approximate voltage drop. The voltage expression is a small-drop per-unit teaching approximation around nominal voltage, not the full three-phase model. Use the arrows to distinguish these quantities before introducing the actual fleet parameter b. The physical tradeoff is that reactive absorption may reduce high voltage while increasing the magnitude of current.

**Sources.** Gan & Low: three-phase distribution optimization (2014); illustrative single-branch linearization, not a feeder measurement

## Frame 45: A small calculation: reactive absorption lowers high voltage

Voltage support: continuous control

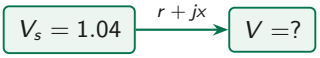
### A small calculation: reactive absorption lowers high voltage

**Illustrative single branch**
All quantities are per unit on consistent bases.

$$V_s = 1.04, \quad r = 0.02, \quad x = 0.04,$$

$$P = -0.60.$$

Negative $P$ means net PV export. $Q$ is net reactive consumption. This is arithmetic for intuition, not output from the Greensboro model.




$$V \approx V_s - rP - xQ.$$

**No reactive absorption:** $Q = 0$
$V \approx 1.04 - 0.02(-0.60) = 1.052$ p.u.
Above the illustrative 1.05 p.u. upper bound.

**Absorb reactive power:** $Q = 0.20$
$V \approx 1.04 + 0.012 - 0.008 = 1.044$ p.u.
The voltage now passes that bound.

Illustrative arithmetic using a linear voltage-drop approximation; Gan & Low: three-phase distribution optimization (2014)

**Next:** Does passing the voltage bound mean the action is acceptable? 45/138

Give the audience time to compute the first voltage before revealing it. The source voltage is 1.04 per unit, the line resistance and reactance are 0.02 and 0.04 per unit, and net active export is 0.60 per unit. With zero net reactive demand, the linear approximation predicts a voltage rise to 1.052. Adding 0.20 per unit of reactive absorption creates a drop that reduces the estimate to 1.044. The example intentionally uses round numbers and a single branch, so it is neither a calibrated feeder result nor a validation of the three-phase adapter. Ask which sign in the equation explains the change. The next slide checks the current consequence of the same action.

**Sources.** Illustrative arithmetic using a linear voltage-drop approximation; Gan & Low: three-phase distribution optimization (2014)

## Frame 46: Voltage passes, but current can still fail

Carry the previous voltage example forward without changing its assumptions. The approximate current uses apparent-power magnitude divided by the fixed source-voltage reference. PV export alone gives acceptable current but excessive voltage. Reactive absorption fixes the voltage while increasing current slightly beyond the illustrative limit. Reducing the magnitude of active export then makes both checks pass in this approximation. The three rows isolate the tradeoff and explain why a good voltage result is not enough. All values are rounded and belong only to this single-branch arithmetic example. They are not the tutorial's actual MPC or SAC commands. Ask participants to name any additional checks still missing, including inverter capability and the validity of the model itself.

**Predict before revealing** Which action would a voltage-only checker mistakenly accept?

**Sources.** Explicitly illustrative per-unit arithmetic; no measured or simulated feeder result

## Frame 47: The five stages turn an action into reviewable evidence

Connect each stage to the physical tradeoff just calculated. Awareness establishes the source conditions, ratings, model scope and permitted actions. Planning proposes a command, using the

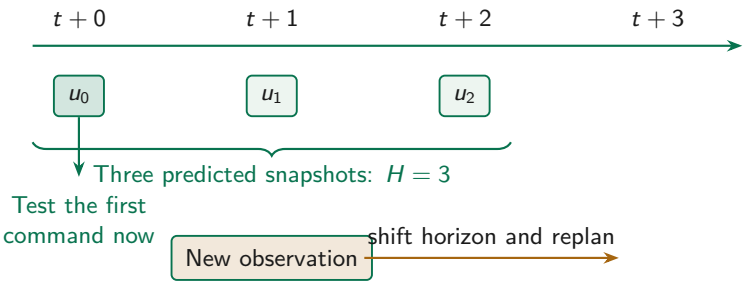
appropriate numerical method. Simulation computes its consequences, while verification decides whether those consequences satisfy the contract. The language optimizer can select a permitted next trial, but the modified command has to return to the numerical stages. The actual tutorial uses MPC and SAC as candidate generators and model-specific power-flow adapters for checking. It does not require an LLM call in every stage. Deterministic computation remains essential. Ask learners to identify which stages repeat after changing only reactive absorption, and which evidence must be replaced.

**Sources.** Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; five-stage proposal architecture

### Frame 48: MPC uses a forecast, then commits only the first move

Voltage support: continuous control

**MPC uses a forecast, then commits only the first move**




$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \gamma^k \ell(x_k, u_k) \quad \text{s.t. prediction model and operating limits.}$$

$u_k$: control; $x_k$: predicted state; $H$: horizon; $\gamma = 0.98$: discount; $\ell$: one-step cost. Here: algebraic power-flow snapshots linked by command-movement cost.

Rawlings, Mayne & Diehl: MPC (2017)

**Next:** Which preferences should one command optimize? 48/138

Build the timeline one element at a time. The central concept is repeated optimization using new information, rather than executing an entire old plan without revision. The generic MPC framework can contain dynamics and long horizons. This tutorial implements H=3 quasi-static power-flow snapshots, linked by a penalty on command movement. It returns the first proposed control for a fresh check. It does not demonstrate transient stability or a complete closed-loop field rollout. Ask learners why a new forecast could change the later controls and why only the first control is considered now. The next slide formalizes these ideas without hiding the difference between general MPC and the implemented approximation.

**Sources.** Rawlings, Mayne & Diehl: MPC (2017)

### Frame 49: The objective makes engineering preferences explicit

Reveal the cost one term at a time and ask learners to name the engineering preference represented by each term. The first term discourages curtailment under the declared availability assumption. The second discourages reactive effort. The third measures mean squared voltage deviation over the modeled bus-phase set. The fourth penalizes command movement. All weights are teaching design choices, not universal operating preferences. Reducing the magnitude of b certainly reduces the reactive-effort term, but can alter voltages, movement and feasibility. The objective is evaluated for the current step in the reported comparisons. A better objective alone establishes neither stability nor a safe recommendation outside the modeled contract.

**Predict before revealing** If $|b|$ decreases, which cost term certainly decreases?

**Explanation** Reactive effort. The voltage and movement terms can still worsen.

**Sources.** Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

### Frame 50: The admissible region is a gate, not another preference

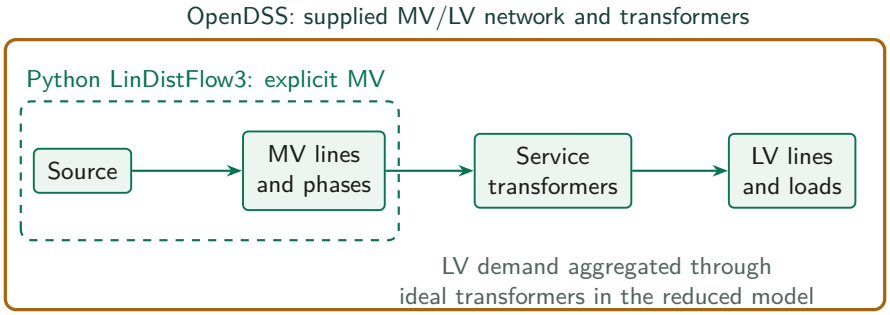
Draw the inverter capability circle while revealing the network bounds. The circle follows from the apparent-power limit after dividing by the squared rating. The planner uses local sensitivities and a conservative capability polygon; the final proposal still returns to a numerical model for checking. Distinguish a solver returning normally, a good cost and a feasible candidate. These are separate observations. Evidence validity also requires matching case, candidate and model identifiers, so results from a previous setting cannot be attached to a new one. Ask whether three language models agreeing can make a branch overcurrent acceptable. The answer locates the engineering gate above preference selection.

**Sources.** Rawlings, Mayne & Diehl: MPC (2017); Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

### Frame 51: The two models cover different parts of the feeder

Voltage support: continuous control

**The two models cover different parts of the feeder**




**Reduced-model PASS**
Within the MV-only approximation and its checked limits.

**Full-model PASS**
Within the broader unbalanced snapshot contract.

**Comparison boundary:** equations *and* constraint coverage change; this is not an accuracy-only test.

Gan & Low: three-phase distribution optimization (2014); EPRI OpenDSS documentation

**Next:** What is retained in the transparent three-phase approximation? 51/138

Keep two model cards visible while adding their coverage. Python LinDistFlow3 implements a coupled three-phase linear approximation on the MV network. Its ideal aggregation does not expose LV voltages or transformer thermal constraints. OpenDSS solves the supplied unbalanced MV/LV power-flow model. Both are useful for their declared purpose, but their PASS labels answer different questions. This experiment changes both model equations and coverage. It therefore cannot isolate a pure approximation-error effect. Even the voltage term in the objective is aggregated over different modeled sets. Ask learners which additional limits could fail when the same command is sent to the broader model.

**Sources.** Gan & Low: three-phase distribution optimization (2014); EPRI OpenDSS documentation

### Frame 52: Three-phase LinDistFlow keeps mutual phase coupling

Start from the parent and child buses, then reveal the voltage-drop term. The equation matches the fixed phase-ratio closure used by the Python adapter. The matrix Z contains mutual phase coupling, so this is not a single-phase equivalent. The beta vector encodes nominal phase relationships, overbars denote complex conjugation, the circle-like multiplication symbol means entrywise multiplication and Re extracts the real component. Branch powers are in VA and squared voltages in volts squared. The imaginary unit in the exponent is distinct from a bus index. Currents use fixed source-reference phasors at 1.03 p.u.; small-loss assumptions are not certified error bounds. The full limitations remain in the handout.

**Sources.** Gan & Low: three-phase distribution optimization (2014); exact adapter: `src/lindistflow3.py`

### Frame 53: SAC learns a continuous action rule; checking remains separate

Use this equation to explain the purpose of SAC rather than derive every update rule. The actor is a probability distribution over continuous actions. The actual saved actor receives three forecast load factors, the previous two-number command, the rating fraction and the remaining-horizon fraction. The task reward expresses operational preferences and penalties, while entropy encourages broader exploration during training. Alpha controls that tradeoff and gamma discounts future rewards. Training and inference are different phases: the classroom loads saved weights and computes a new action for the current observation. It does not retrain the policy. Even a strong reward penalty for violations can leave unsafe actions under distribution shift. This is why the actor is a candidate generator within the workflow, followed by simulation and a hard acceptance gate.

**Sources.** Haarnoja et al.: SAC, ICML 2018; Schulman et al.: PPO (2017)

### Frame 54: Training provenance prevents a misleading speed claim

Animate a pipeline from AC samples to surrogate fitting to policy training to the saved actor. The 12,000 transitions are not 12,000 OpenDSS solves. This distinction matters for reproducibility and for reporting computational cost. The fit-validation samples assess the surrogate, not all aspects of policy robustness. New classroom scenarios can alter demand or current-rating conditions, creating a transfer problem for the frozen actor. A quick inference time therefore says little about total development cost or reliability under shift. Ask learners what evidence they would request before trusting the actor beyond this fixture. Possible answers include separate policy test cases, out-of-distribution conditions, constraint failures and an appropriate deterministic baseline.

**Sources.** Haarnoja et al.: SAC, ICML 2018; executed training artifact: `Continuous_SAC_Training.ipynb`

### Frame 55: L2 recomputes both proposals under one declared scenario

Walk through the exact first seven lines of L2. LOAD_SCALE and RATING_FRACTION are the two visible scenario controls. The helper performs a fresh MPC calculation, new saved-actor inference and new numerical checks. The timer encloses the helper call, so it measures more than policy inference alone. The returned table is the place to compare feasibility and cost before looking at the plot. Keep the original default outputs as a reference, then make one change at a time. The later L3 response pool belongs to a different archived original context; rerunning L2 does not regenerate those LLM responses. This boundary must stay explicit during the exercise.

**Notebook excerpt.** Run the complete cell in context.

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE, RATING_FRACTION)
timings['L2 fresh MPC + SAC + checks'] = time.perf_counter() - started
print(voltage['mode'])
display(voltage['table'])
```

**Sources.** Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

### Frame 56: Lab: predict, change demand, then explain the response

Allow seven minutes and keep this slide visible while participants work. One learner predicts and one operates the notebook, then they exchange roles. First record the default results. Next choose a modest changed load multiplier and predict how PV participation, reactive support or curtailment might respond. Run the full L2 cell and compare the same acceptance metrics. Students should explain a result using a numerical margin rather than a general claim about AI. If setup fails, use the executed notebook output to complete the prediction and interpretation tasks. Do not modify unrelated cells or imply that the archived LLM pool has changed. Finish by collecting one surprising observation from the room.

**Predict before revealing** Must higher demand increase PV curtailment?

**Explanation** No. Voltage and current constraints can move in different directions.

**Notebook excerpt.** Run the complete cell in context.

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE, RATING_FRACTION)
```

**Sources.** Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

### Frame 57: Inspect the physical margins before ranking the controllers

Use the actual executed L2 figure. The displayed default costs are rounded for teaching: MPC is approximately 0.031104 and SAC 0.436428, with both feasible under the reduced-model contract. The voltage curves are sorted separately, so matching positions are ranks rather than the same physical bus. The current-utilization plot reports the most heavily used allowed current bound. Ask participants to identify the evidence for feasibility before comparing objective values. A cost gap here is a controller result, not automatically an advantage of the multi-agent architecture. Students with changed load settings should report their own values and avoid mixing them with this original reference output.

**Sources.** Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

### Frame 58: L3 opens a different, explicitly archived context

Explain why this cell is reproducible without contacting a local model. It loads an archived response pool from original test_01, displays each exact public response and reports how the structured proposals were canonicalized. The seconds column contains recorded call durations, not the time required to print the table. The original test context is not identical to L2's default rating fraction, and certainly not to a student-modified L2 scenario. This cell therefore teaches inspection and

evidence attribution. It does not supply new context-matched recommendations for the latest classroom perturbation. Ask learners to find a claim in a reason that is supported by the numeric evidence and one that still requires testing.

**Notebook excerpt.** Run the complete cell in context.

```
pool = lab.saved_pool('test_01')
display(pd.DataFrame(pool['responses'])[['model_id', 'operation_id', 'reason', 'seconds', 'source'
    ]])
display(pd.DataFrame([{'operation': ', '.join(c['operation_ids']),
    'votes': c['vote_count'], 'share': c['vote_share'], 'parameters': c['operation']['parameters'
    ]}
    for c in pool['pool']['candidates']]))
print('ARCHIVED ACTUAL LLM; original context:', pool['context']['id'])
print('Distinct physical candidates:', len(pool['pool']['candidates']))
print('Malformed/duplicate votes rejected:', len(pool['pool']['rejected']))
```

**Sources.** Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

### Frame 59: Read the actual replies, then challenge their claims

These times and choices come from the recorded response-selection pilot. Llama 3.1 suggested increasing active participation to reduce curtailment. Llama 3.2 described reducing reactive effort and also mentioned reducing curtailment, a causal connection that its text alone does not establish. Qwen used the phrase voltage stability, which is unsupported by these steady-state tests. Present the excerpts as public rationales, not access to private model reasoning. Llama names the model family; Ollama is the local runtime that served it. Have the audience annotate the difference between a permitted operation and a justified physical claim. The next step converts prose-backed choices into reproducible physical candidates.

**Predict before revealing** Which phrases need evidence beyond a plausible explanation?

**Sources.** Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

### Frame 60: Canonicalize actions before counting votes

Animate three response cards merging into two physical-candidate cards. Two different descriptions of the same command must not count as two distinct control actions. Canonical identifiers depend on the tool and parameters rather than rhetorical style. In original test_01, two models propose the same reactive reduction and one proposes a PV increase, so the majority chooses the reactive trial. A predetermined hash tie-break makes ties reproducible; it is not an engineering preference. Agreement is also not a calibrated safety probability because models can share mistaken assumptions. A changed command returns to the numerical stages. Ask what should happen if the majority-selected trial fails a current constraint.

**Sources.** Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

### Frame 61: LinUCB learns which proposed operation is worth testing

Explain select, observe and update before revealing the formula. The feature vector represents the known context and a canonical operation, including vote-related information. Theta stores learned reward coefficients; A stores regularized feature information. During training, beta controls an uncertainty bonus. After a trial, selected feedback updates the reward model. The classroom evaluation freezes these parameters and uses predicted reward only, with no test updates. All candidate LLM responses have already been generated, so their call costs are paid by every pool

selector. This is a contextual bandit, a simple reinforcement-learning setting, rather than training the language models or adapting a full sequential control policy online.

**Sources.** Li et al.: LinUCB, WWW 2010; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

### Frame 62: Reward learning and physical acceptance have different jobs

Read the reward in two branches. A passing trial can earn a bounded improvement reward relative to the incumbent cost, with a fixed trial charge. A rejected trial receives a negative reward. The 0.01 scaling and charge are design choices; the charge does not measure latency. Training consumes selected outcomes, although the audit records contain other outcomes for comparisons. Validation and test are frozen, avoiding feedback leakage. Five training orders reuse the same training contexts and are not five independent datasets. Ask learners how changing reward scaling might change selection without changing any electrical operating limit. This separates learning preferences from the hard verification contract.

**Predict before revealing** The learner predicts improvement, but the trial violates current. Accept?

**Explanation** Reject the trial. Keep a valid incumbent or escalate; reward does not grant permission.

**Sources.** Li et al.: LinUCB; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

### Frame 63: L4: verify the frozen choice with a new numerical trial

Read the exact L4 code, then give participants three minutes. The stored selector chooses an operation using its frozen parameters. The numerical helper evaluates the selected trial again under the archived original context. The displayed predicted reward is a learned estimate, while the returned trial table contains newly calculated outcomes. Participants should check whether the canonical candidate matches the majority choice and whether it passes the engineering gate. In this recorded test, the learner and majority agree on reactive reduction. Do not subtract this cost from a modified L2 cost, because those are different contexts. Ask which outcome controls the recommendation if a predicted improvement conflicts with failed verification.

**Notebook excerpt.** Run the complete cell in context.

```
decision = pool['selection']
print('Frozen:', pool['frozen'], '| selected-feedback training updates:', pool['training_updates'
    ])
print('Bandit operation:', decision.operation_id, '| predicted reward:', round(decision.
    predicted_reward, 4))
print('Same canonical choice as majority:', decision.candidate_id == pool['vote_id'])
started = time.perf_counter()
selected_trials = lab.trial_saved_selection(pool)
timings['L4 new verification of archived choices'] = time.perf_counter() - started
display(selected_trials['table'])
```

**Sources.** Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

### Frame 64: A fair experiment can find no learned advantage

Present this result without framing a tiny cost change as general superiority. The majority vote and frozen learner tie on all twelve held-out MV cases. A simple posthoc rule that always selects the

reactive reduction reproduces those choices without language-model calls. Because that rule was identified after inspecting results, label it a diagnostic rather than a predeclared baseline. The experiment demonstrates that bounded proposals, fresh verification and reproducible traces can be implemented. It does not demonstrate that additional LLMs or a learned selector are necessary for better control. Ask what new test set or task distribution would make the learner's contribution more convincing than the simpler supervisor.

**Sources.** Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

### Frame 65: Report the latency of the entire decision path

The two reported times are reconstructed serial accounting from actual recorded components in the held-out pilot. They are not a guaranteed deployment response time and should not be confused with a new classroom timing measurement. Pool-based selectors share the cost of generating the three responses. The learned selector's small scoring operation is only one component of that total. Cold model loading, prompt processing, token generation and numerical checking can have very different scaling. Ask learners to draw the boundary around the quantity they want to compare. The engineering question is whether any incremental benefit justifies the additional decision latency under the intended operator workflow.

**Sources.** Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`; Ollama chat API timing fields

### Frame 66: L5 separates same-action auditing from AC replanning

Read the helper call before enabling its optional path. The same-action audit asks whether an existing recommendation survives a broader physical model. The separate AC MPC solve asks what command a planner using that broader model would propose. Mixing those questions can make model coverage look like an LLM improvement. The default classroom execution performs the MV check; enabling RUN_OPENDSS in L0 requests the fresh AC audit and separate AC replanning in L5. The saved learner remains frozen. OpenDSS computations should be performed sequentially because the local engine has shared mutable state. Compare the status, limiting quantities and model scope rather than treating cross-model costs as an exact solver ranking.

**Notebook excerpt.** Run the complete cell in context.

```
started = time.perf_counter()
scope_audit = lab.backend_audit(pool, run_opendss=RUN_OPENDSS)
timings['L5 selected model scope checks'] = time.perf_counter() - started
display(scope_audit['table'])
print(scope_audit['optional_mode'])
print('An MV-only PASS cannot acknowledge a request requiring full MV/LV coverage.')
```

**Sources.** Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; EPRI OpenDSS documentation

### Frame 67: A reduced-model PASS can fail the full-feeder audit

The table summarizes the separately recorded twelve paired scenarios, not the output of one L5 cell. The reduced model accepts all twelve final actions under its own scope, yet their same-action full-feeder audits fail. Planning with the AC backend returns actions that pass the full-feeder snapshot contract in these scenarios. The result is caused by the broader model and constraint coverage, with the selector still frozen. It cannot be attributed to more convincing Llama explanations. Within the AC pilot, the learned selector also does not establish superiority over fixed or uniform selection. Ask learners how a responsible operator packet should describe an MV-only recommendation when full MV/LV coverage was requested.

**Sources.** Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

### Frame 68: The AC comparison still needs a strong numerical baseline

The broader AC model improves the relevance of the checked constraint set, but that does not establish a benefit from the learned response selector. In the twelve paired AC scenarios, the frozen learner and majority vote tie, and their mean current-step cost is worse than the fixed-refinement and uniform-pool comparators. The raw SAC proposal passes only one of twelve AC checks in this pilot, illustrating transfer limitations. Explain that every listed final recommendation still needs its declared physical gate. Do not compare the AC mean directly with the MV mean as if they measured an identical voltage set. A rigorous architecture study isolates the contribution of each changed component and reports negative results alongside successes.

**Sources.** Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

### Frame 69: The operator receives a scoped recommendation and its evidence

Close the voltage chapter with the complete path and the actual findings. The numerical controllers supplied proposed settings. The language models supplied bounded refinement choices and public explanations. Voting and frozen learned selection tied in the held-out MV pilot. Broader-model checking revealed that the reduced-model recommendations did not meet the requested full-feeder scope. These are valuable workflow lessons without a claim of universal agentic control superiority. The operator must see the command, scenario, evidence, failed checks, remaining assumptions and required scope. Readback remains advisory and does not dispatch equipment. Invite one participant to name a demonstrated benefit and another to name a limitation, then take the scheduled ten-minute break before restoration.

**Sources.** Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`, L1–L5 and L12; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

# 3 Break

### Frame 70: Ten-minute break: 80–90 minutes

Take the full break. Participants do not need to install packages or run a new cell during it. Before leaving the voltage chapter, ask them to keep a note of the action, which model checked it, and which limits were evaluated. At the restart, recall that inverter settings are continuous decisions. Restoration introduces discrete load pickup and network choices. That contrast creates a reason for a different planning method instead of assigning MPC to every problem. The next case will make the priorities visible using six numbered nodes, a failed connection and two optional ties.

**Predict before revealing** Would the largest number of restored kilowatts always be the right objective?

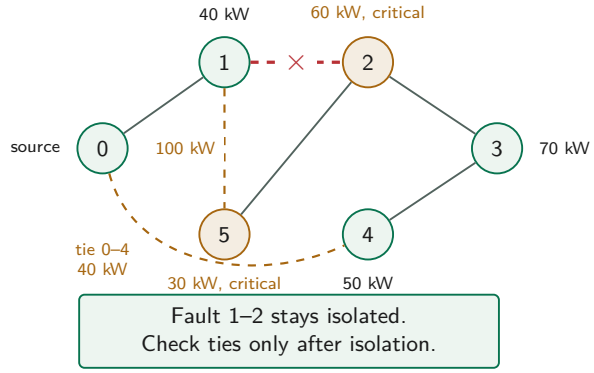
**Sources.** Workshop timetable. Take the full ten-minute break.

# 4 Restoration

## Frame 71: A fault changes the paths that are available

Restoration: paths, capacity and priorities

### A fault changes the paths that are available




**A small, transparent fixture**

Six nodes; five loads; one failed segment; two optional ties. Amber nodes have higher service priority.

**Predict:** Can we simply reconnect the failed segment?

**Explanation:** No. Its availability is fixed to zero. Planning must find another admissible path.

Fresh classroom fixture: Classroom_Labs L6; src/standard_formulations.py. Restoration context: Chen et al., IEEE TPWRS, 2018.

**Next:** What can each alternative path carry? 71/138

Begin with the physical picture, not an optimization acronym. Node zero is the available source. The interrupted segment connects nodes one and two, and is unavailable throughout the experiment. The dashed amber links are candidate ties, not commands already issued. Nodes two and five are designated critical in this classroom fixture. Explain that the drawing is a six-node active-power transport example, not a reduced drawing of the full Greensboro feeder. Its purpose is to expose the decisions clearly enough to check them by hand. Ask the audience to point to the failed segment and identify a route that avoids it. Reveal the isolation statement before discussing any restoration action. A real switching procedure additionally needs protection, equipment status, work permits and operator authority, which this static fixture does not establish.

**Predict before revealing** Can we simply reconnect the failed segment?

**Explanation** No. Its availability is fixed to zero. Planning must find another admissible path.

**Sources.** Fresh classroom fixture: Classroom_Labs L6; src/standard_formulations.py. Restoration context: Chen et al., IEEE TPWRS, 2018.

## Frame 72: Two ties create choices, with different bottlenecks

Trace the teal route with a pointer while it appears. The hundred-kilowatt tie between nodes one and five must carry the downstream demand that the chosen tree places behind it. Node one's own forty-kilowatt demand can be supplied before that bottleneck. The other tie supplies the lower side of the diagram, but at forty kilowatts it cannot pick up the fifty-kilowatt block at node four. That conclusion relies on the all-or-nothing pickup assumption. It would change if loads could be divided into smaller controllable groups. The colors describe routes for reasoning; the slide does not certify a safe temporal switching sequence. Ask participants to distinguish a branch rating from the total rating of the source. The next distinction is between energizing a bus so power can pass through it and reconnecting that bus's local demand.

**Predict before revealing** Can the 40 kW tie pick up the entire 50 kW load at node 4?

**Sources.** Fixture and binary pickup assumption: Classroom_Labs L6; standard_formulations.restoration_graph. Blue and teal paths are explanatory alternatives, not a switching sequence.

### Frame 73: A node can carry power while its load stays disconnected

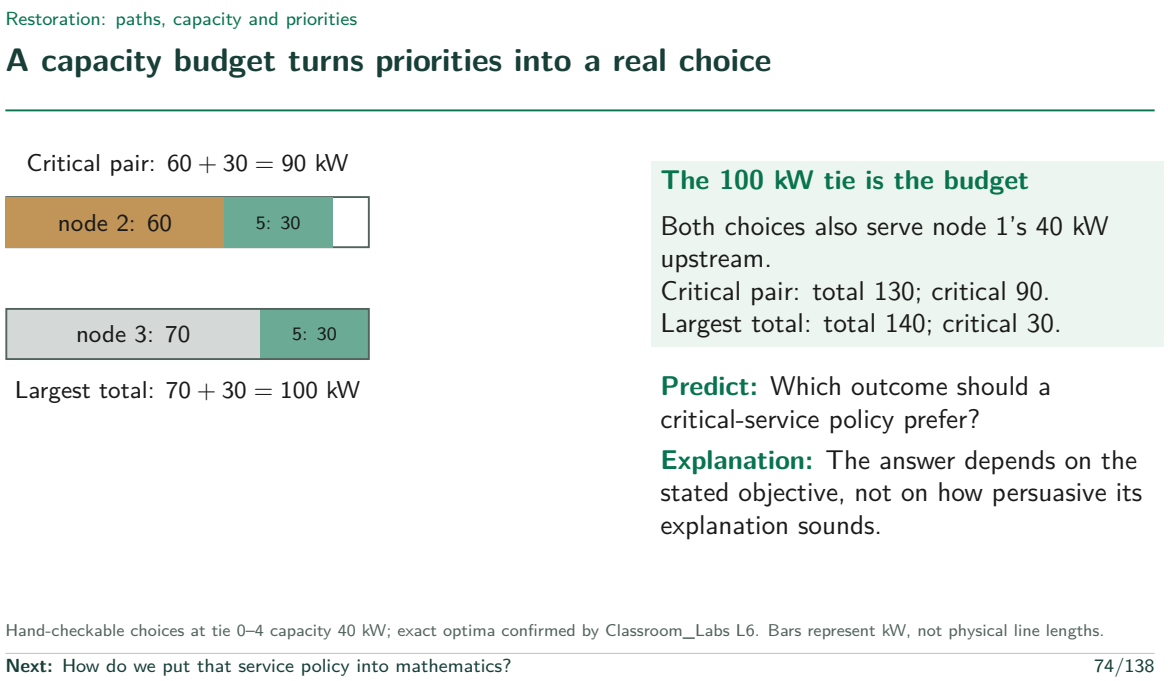
This is an important modeling distinction that is easy to miss when a diagram uses one color for everything. The illustrated path can carry power through node two while leaving its sixty-kilowatt demand disconnected. In the example, the tie between one and five carries thirty kilowatts for node five plus seventy for node three, exactly one hundred kilowatts. Node one's forty kilowatts are supplied upstream. The total picked up is therefore one hundred forty kilowatts. Define the energized variable and the pickup variable in words before showing their inequality. The figure is an explicit feasible explanation of the unweighted service level, not a claim that every optimizer returns this exact edge list. Different optimal trees can exist. Invite participants to explain why a transit node still needs valid energized connectivity even when its local pickup variable is zero.

**Predict before revealing** Why is equality $s_i = y_i$ too restrictive?

**Explanation** It would forbid a useful transit node whose own load remains shed.

**Sources.** Original schematic of a feasible six-node plan; binary pickup/energization model in src/standard_formulations.py.

### Frame 74: A capacity budget turns priorities into a real choice




Use the bars as a budget, with each indivisible load consuming part of the hundred-kilowatt tie capacity. The top choice carries the two critical loads, sixty plus thirty kilowatts, leaving ten unused. That unused capacity cannot serve any remaining complete load block. The bottom choice uses all one hundred kilowatts to carry seventy plus thirty. Both plans also serve forty kilowatts at node one upstream, so their total service is one hundred thirty and one hundred forty respectively. Ask for a preference before revealing the policy interpretation. The lower total service is not evidence that the weighted optimizer failed. It can be the intended consequence of protecting critical demand. Equally,

the higher total service is not a mistake under an explicitly unweighted objective. This prepares a fair comparison: same feasible network, transparent objectives, and both total and critical service reported.

**Predict before revealing** Which outcome should a critical-service policy prefer?

**Explanation** The answer depends on the stated objective, not on how persuasive its explanation sounds.

**Sources.** Hand-checkable choices at tie 0–4 capacity 40 kW; exact optima confirmed by Classroom_Labs L6. Bars represent kW, not physical line lengths.

### Frame 75: Compare outcomes before comparing algorithms

Keep the table visible long enough for participants to read both columns. Reporting only total service would conceal the reason the weighted formulation was chosen. Reporting only critical service would conceal the sacrifice in ordinary service. The first comparison intentionally changes the objective, so it is a policy comparison. It is not a controlled test of language-model orchestration. The unweighted baseline is solved exactly by enumeration on the same small network and constraints; it is not a deliberately weak random baseline. The weighted MILP is also checked against exhaustive enumeration. Later we will compare a fixed supervisor and an LLM supervisor with matched objectives and tools, using a separate archived feeder experiment. Ask the audience to list what must be held equal in that later comparison. This distinction helps them avoid attributing every favorable result to the framework.

**Predict before revealing** What must stay matched to isolate an agentic workflow effect?

**Explanation** The objective, candidate tools, physical limits, information and verification rules.

**Sources.** Fresh six-node experiment: Classroom_Labs L6, weighted MILP versus unweighted exact enumeration; both use the same feasible constraints.

### Frame 76: The objective is an explicit, inspectable service policy

Read the expression aloud as demand multiplied by a priority and a pickup decision, then summed over loads. Distinguish actual kilowatts from the weighted score. The critical-pair plan earns nine hundred forty weighted kilowatts, while the illustrated maximum-total plan earns four hundred ten. The factor ten is an instructional policy choice, not an empirical estimate of a hospital's value or an industry-wide standard. A finite weight can be outweighed by sufficiently much ordinary demand, so it does not automatically encode strict critical-first restoration. If strict priority is required, solve critical service first and preserve that optimum in a second objective, or justify a bound on the weight. The planner should expose this assumption to the operator rather than letting an LLM silently invent priorities. Now move from scoring plans to checking which plans are feasible.

**Predict before revealing** Does weight ten guarantee critical-first service on every possible network?

**Explanation** No. Strict priority requires a lexicographic objective or a proven sufficiently large weight.

**Sources.** Classroom priority policy: standard_formulations.restoration_graph. Weighted restoration objectives: Chen et al., 2018.

### Frame 77: Planning: binary pickup and topology decisions

Introduce one decision type at a time and connect it to the preceding diagrams. The pickup variable describes demand, the energized variable describes the usable network, and the edge variable describes the selected topology. Signed flow needs an arbitrary reference orientation; a negative value simply means power travels the other way. The source is fixed energized, and the failed segment is fixed open. The endpoint constraints prevent a selected line from attaching to a node labeled de-energized. These are necessary logical conditions, but they do not prove that every energized node is reachable from the source. Pause on that omission. An isolated cycle could satisfy many local conditions while being impossible to supply. The next two slides add active-power transport and source connectivity. This modular construction explains what each constraint contributes rather than presenting one intimidating formulation without interpretation.

**Predict before revealing** Which of these rules prevents a disconnected energized island?

**Explanation** None yet. Connectivity must be imposed separately.

**Sources.** Fresh MILP implementation: src/standard_formulations.py. Radiality and connectivity: Wang et al., 2020.

### Frame 78: Power balance says where every picked-up kilowatt comes from

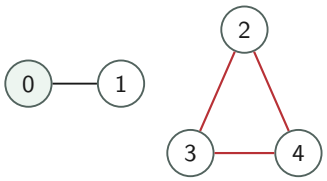
Use the small flow diagram to show that balance is bookkeeping with a physical interpretation. One hundred kilowatts arrive at node five, thirty are consumed locally, and seventy continue downstream. With the stated incidence convention, incoming flows enter positively and outgoing flows negatively. At an open edge, the binary variable is zero, forcing its flow to zero. At a closed edge, the same inequalities bound the magnitude by the rating. The source balance is handled as the supply needed to satisfy non-source demand. State the model limitation clearly: a feasible active-transport solution is not an AC security certificate. It ignores voltage, reactive power, losses and protection behavior. That simplicity is deliberate for a hand-checkable lesson. The later archived feeder result uses additional power-flow evidence, but must not be confused with this exact MILP fixture.

**Sources.** Classroom_Labs L6; active-transport formulation in src/standard_formulations.py. This six-node model is separate from the archived feeder power-flow checks.

## Frame 79: Edge counts alone do not prove a source-connected tree

Restoration: paths, capacity and priorities

### Edge counts alone do not prove a source-connected tree




5 nodes; 4 edges
but an unsupplied cycle

$$\sum_e A_{ie} f_e = y_i \qquad (i \neq 0),$$

$$-5z_e \leq f_e \leq 5z_e,$$

$$\sum_e z_e = \sum_i y_i - 1.$$

$f_e$: fictitious connectivity flow, not electrical kW. Every energized non-source node consumes one unit. Five is the maximum number of such nodes.

**Two jobs, two conditions**

Commodity flow proves reachability. Reachability plus the edge count gives a tree on the energized nodes.

Original counterexample; single-commodity connectivity and radiality: Wang et al., 2020; classroom MILP.

**Next:** How do we check a small MILP without trusting its solver blindly? 79/138

The left-hand counterexample is a separate five-node sketch, not the six-node restoration fixture. It has the familiar tree edge count but is disconnected: the source supplies one short component while three other nodes form a cycle. This makes the missing condition visible. Reveal the fictitious flow equations and explain that these units are proof tokens. Every energized non-source node must receive one token through selected edges, so an island without the source cannot satisfy the summed balances. The source provides the required tokens. The bound of five is enough because the classroom network has five non-source nodes. Once all selected energized nodes are connected, exactly one fewer edge than nodes eliminates cycles. Emphasize that this proves a static graph property. It does not address the time ordering of switching, transient stability or protection coordination.

**Sources.** Original counterexample; single-commodity connectivity and radiality: Wang et al., 2020; classroom MILP.

## Frame 80: Verification uses an independent, exhaustive small-case check

Explain what independent means here. The enumeration routine loops over candidate edge sets and pickup choices, checks source reachability, the tree property and branch loading, and computes the service objective. It does not simply ask the MILP solver whether its own answer is acceptable. Agreement provides a strong check for this finite small fixture, although it does not prove that the simplified physical model covers every operational hazard. The second optimization pass preserves the best weighted service and prefers fewer energized edges, with a small index-based tie-breaker. Avoid calling that minimum switching: without initial switch positions and a temporal sequence, those are different quantities. The exhaustive method scales poorly, which is why it belongs in a small educational verification exercise. Ask participants to predict the forty-kilowatt result before running the sweep, so the computation tests a reasoned expectation.

**Predict before revealing** Does minimizing selected edges minimize switching operations?

**Explanation** No. Operation count needs the initial switch states and a sequence-aware model.

**Sources.** Classroom_Labs L6; standard_formulations.solve_restoration_milp and _enumerate_restoration.

### Frame 81: Hands-on: vary the bottleneck, keep the policy visible

Give participants the full eight minutes rather than using the lab slide as another lecture slide. Ask them to run the entire L6 cell, which includes the plot and the assertion that the weighted MILP matches the independent exhaustive check. The visible five lines identify the parameters and the result table. Participants should record total pickup, critical pickup and the measured runtime from their own environment. The runtime in this cell includes the fresh sweep and independent checks; it is not a single MILP solve or LLM latency. If someone changes the capacity samples, have them keep a note of the change so comparisons remain interpretable. The key explanation is a plateau: binary loads come in blocks, so extra capacity may remain unusable until it is large enough to connect another whole load.

**Notebook excerpt.** Run the complete cell in context.

```
CAPACITIES_KW = (40., 60., 80., 100., 120.)
started = time.perf_counter()
restoration = lab.restoration_sweep(CAPACITIES_KW)
timings['L6 fresh MILP sweep + independent enumeration'] = time.perf_counter() - started
display(restoration['table'].drop(columns='pickup'))
```

**Sources.** Exact excerpt: Classroom_Labs.ipynb, L6 lines 1–5. Fresh results use the six-node active-transport fixture.

### Frame 82: More capacity can produce the same discrete pickup

Point first to the axes and the discrete sample markers. The plot compares policy choices at a set of tie capacities, not a continuously optimized physical design. At eighty and one hundred kilowatts, the weighted solution serves two hundred kilowatts in total and all ninety kilowatts of critical demand. A rating increase does not always admit another indivisible load block. The figure also lets participants see where objectives coincide, which is as important as the point where they differ. Do not interpret coincident markers as evidence that only one method ran. Both outcomes are plotted. Ask a participant to relate one marker back to a feasible path and the branch bottleneck from the earlier diagrams. Finish by distinguishing optimization quality from workflow quality: the next slide shows how the calculation becomes a traceable proposal, with assumptions and checks attached.

**Predict before reveling** Is the flat step evidence that capacity has no value?

**Explanation** No. It reflects indivisible load blocks and the available routes at these sampled ratings.

**Sources.** Original computed figure: Classroom_Labs L6 / classroom_restoration.png. Same feasible constraints; weighted versus unweighted objectives.

### Frame 83: The workflow surrounds the solver with evidence and decisions

Read each row as a method producing a concrete output, rather than as five people having a conversation. Awareness binds the request to a topology and trusted ratings. Planning chooses pickups and edges. Simulation evaluates the transport consequences. Verification checks feasibility and, in this small case, agreement with an exhaustive optimum. The LLM optimizer may propose an allowed refinement or explain a tool result, but it does not override a failed capacity check. Be precise about what students are executing. L6 and L7 provide fresh numerical and evidence-binding experiments. L10 displays the full archived five-stage trace from a different feeder fixture. It would be misleading to imply that L6 itself makes all those language calls. The common framework is the contract among stages, while the physical models and numerical methods vary with the task.

**Sources.** Classroom_Labs L6–L7, L10; proposal_complete_workflow.json. This table describes stage roles; L6 does not make five live LLM calls.

### Frame 84: Hands-on: old evidence does not certify a changed rating

This exercise tests a workflow failure that can occur even when the original optimizer worked perfectly. The request changes from a forty-kilowatt tie rating at revision one to twenty kilowatts at revision two. The previous candidate and its evidence remain associated with the original request. The binding check should return true for that original request and false for the changed request. That failure means the old result is not valid evidence for the new contract; it does not independently prove that the old pickup is physically infeasible. A new constrained solve is required to decide that. Ask participants to name other changes that should invalidate evidence, such as topology, priorities, measurement provenance or the model version. The intended benefit is preventing stale recommendations from being presented as current checked actions, not improving the MILP objective.

**Explanation** Original binding: True. Changed request with old evidence: False. HOLD and re-solve.

**Notebook excerpt.** Run the complete cell in context.

```
new_rest_request = {**rest_request, 'capacity_kw': 20., 'revision': 2}
print('Original contract:', lab.bound_to(rest_request, rest_candidate, rest_evidence))
print('Changed rating with old evidence:', lab.bound_to(new_rest_request, rest_candidate,
    rest_evidence))
print('Required response: HOLD old recommendation; request a fresh constrained solve.')
```

**Sources.** Exact excerpt: Classroom_Labs L7 lines 7–10; lab.bind_evidence and lab.bound_to. Binding checks identity, not physical feasibility by itself.

### Frame 85: Archived feeder: an attractive pickup can fail a thermal check

Make the fixture change explicit before presenting the numbers. We have left the six-node classroom graph and are inspecting an archived medium-voltage feeder workflow. It has a synthetic bypass catalog and an ideal low-voltage aggregation assumption. In the LinDistFlow3 run, the initial high-pickup candidate overloads the bypass to about one hundred fifty-two percent of its rating. That numerical result triggers rejection regardless of its language explanation. A permitted refinement sheds a specified fraction of ordered noncritical load objects, after which a candidate passes the modeled constraints. The final served and critical-service values belong only to this archive. They should not be combined with the six-node optimum table. The trace demonstrates rejection, refinement and rechecking. It does not establish a safe field switching sequence, protection coordination or a deployed operator decision.

**Sources.** ARCHIVED ACTUAL: proposal_complete_workflow.json, restoration / lindistflow3. Initial E01 loading 151.639%; final verified SHED_075.

### Frame 86: Llama can request a useful check and still rank results incorrectly

Read the planning response exactly as recorded, without presenting it as hidden chain-of-thought. It is an observable rationale for requesting tools. The call took about eight hundred seventy-four milliseconds in the archived environment. The later language ranking provides a useful counterexample to treating fluency as numerical reliability. The returned sequence places E02 before E03 even though the objective is minimized and E03 has the more negative score. A deterministic ranking guard corrects that ordering using verified values. Multiple similar LLM votes would not make the incorrect comparison true. This is a concrete workflow benefit: language proposes and explains, while explicit numerical rules constrain acceptance. It is also a limitation of the language component. The final candidate receives fresh verification, so acceptance is attached to evidence rather than to a persuasive ranking statement alone.

**Predict before revealing** Should a majority of similar language responses overrule the score?

**Sources.** Exact Planning quote and returned ranking: archived restoration / lindistflow3 trace, proposal_complete_workflow.json. Times are recorded model-call wall times.

### Frame 87: Matched supervision: equal service, different latency

This is the controlled comparison within the archived restoration setup. Both supervisors use the same final physical task, candidate tools, constraints and cost definition. They reach the same final service and objective. The language-driven supervisor takes substantially longer in this single recorded run. Avoid describing the ratio as a general benchmark; model loading, hardware, caching and task difficulty can change latency. The trace is useful for seeing what was requested, why the high-pickup candidate failed, and how a later candidate was checked. That usefulness must be measured separately from electrical performance. Suggest future matched evaluations involving changed ratings, failed tools and stale evidence, with explicit acceptance rules and operator workload measures. Do not imply these tests already demonstrate superiority. The honest conclusion is that this case exposes workflow behavior and limitations while preserving the conventional numerical result.

**Predict before revealing** What would demonstrate an additional agentic benefit?

**Explanation** Matched tests of changed requests, tool failures and stale evidence, with success rate and human effort measured.

**Sources.** ARCHIVED ACTUAL: fixed versus agentic, restoration / lindistflow3, proposal_complete_workflow.json. Single recorded case; four Llama calls.

### Frame 88: Handoff: show the action, the evidence and the remaining decision

Close restoration with the operator's view. A recommendation should identify what demand is restored, what topology it assumes, the service priorities, the modeled limits and the evidence revision. It should also state what was not certified, especially switching sequence and protection behavior in these exercises. Acknowledgment is not merely clicking through a green badge; it is a decision made with those conditions visible. Participants have now seen two complementary benefits: an optimization objective that protects designated demand, and a workflow that prevents stale or failed evidence from being treated as valid. Neither requires claiming that Llama improves the optimum. Use the final question as the transition to cyber response. Even a correct network model and a competent optimizer can recommend the wrong action when measurements describe the wrong state. The next lesson separates consistency from trust.

**Predict before revealing** What if the network model is correct but the measurements are deceptive?

**Explanation** The planner may optimize the wrong state. The next case tests consistency, observability and trusted replacement evidence.

**Sources.** Restoration lessons: Classroom_Labs L6–L7 and archived L10 trace. Operator acknowledgment is a tutorial workflow, not a field execution system.

# 5 Cyber response

### Frame 89: Sensors report numbers; the state is what we infer

Start by distinguishing the network state from the sensor readings. In this small DC example, the state consists of three voltage angles relative to the reference bus. Voltage magnitudes and reactive power are outside the model. The measurements include branch flows, nodal injections and one angle reading, so they provide more equations than unknowns. That redundancy makes it possible to ask whether the readings agree with one modeled state. The arrows define branch reference directions, not necessarily actual positive power flow. Do not begin with an attack mechanism. First establish what an ordinary estimator is trying to do and what its residual means. The next two-sensor example is a separate arithmetic illustration, intentionally much simpler than this four-bus model. It lets the audience predict the estimator output before seeing matrix notation.

**Predict before revealing** Is a small residual the same as a trustworthy state?

**Explanation** No. We will construct two different states with the same residual.

**Sources.** Classroom_Labs L8–L9; standard_formulations.run_wls_demo. State estimation: Schweppe and Wildes, 1970.

### Frame 90: Two equally precise sensors: find their common explanation

Ask the audience for the best common explanation of nine and eleven before revealing ten. Assume both sensors measure the same scalar in the same units, with equal independent noise variances of one. Under squared-error fitting, the average minimizes the sum of the two squared discrepancies. A residual is the observed reading minus the reading predicted by the fitted state. The two residuals are minus one and plus one; their squared sum is two. Explain the hat notation as an estimate rather than a true value. No detection threshold is being claimed for this arithmetic example. Its purpose is to make the estimator and residual concrete. The actual classroom network has different measurement types and variances, which later require weighting so an angle discrepancy and a flow discrepancy are compared in their own noise scales.

**Predict before revealing** What mismatch remains after choosing the best common value?

**Explanation** $J = (-1)^2 + 1^2 = 2$. A residual measures mismatch to the fitted model.

**Sources.** Original arithmetic teaching example; least-squares principle: Schweppe and Wildes, 1970.

## Frame 91: An isolated error increases disagreement between sensors

Cyber response: consistency and trustworthy evidence

### An isolated error increases disagreement between sensors

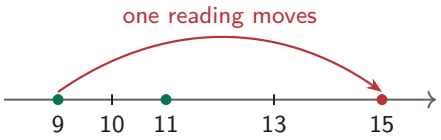
**Before corruption**

$z = [9, 11]^T$
$\hat{x} = 10$
$r = [-1, 1]^T$
$J = 2.$

**Change only sensor 1**

$z' = [15, 11]^T$
Which common estimate fits best?

$$\hat{x}' = 13, \quad r' = [2, -2]^T, \quad J' = 8.$$




A calibrated alarm signals model inconsistency; it does not identify an attacker.

Original scalar illustration, equal unit variances. Statistical alarm assumptions are introduced on C06.

**Next:** What if both readings move together? 91/138

Keep the true physical quantity conceptually unchanged while modifying only the first reported number. The new readings, fifteen and eleven, are best explained by thirteen. Their residuals are plus two and minus two, so the squared mismatch rises from two to eight. This provides an intuitive reason why redundant measurements can detect some kinds of corruption: the readings no longer agree with one another as well. Do not label this arithmetic result an actual alarm, because we have not chosen a statistical threshold for this example. A larger residual can also arise from a failed sensor, incorrect topology, poor calibration or an inadequate model. An alarm indicates inconsistency under assumptions, not malicious intent. Ask what happens if both readings are changed by the same amount. That prediction prepares the central limitation without requiring matrix algebra yet.

**Sources.** Original scalar illustration, equal unit variances. Statistical alarm assumptions are introduced on C06.

## Frame 92: Coordinated errors can preserve the same disagreement

This is the explicit counterexample the audience should remember. Both sensors are shifted by three, so their best common explanation shifts from ten to thirteen. Their differences from that explanation remain minus one and plus one. The residual test therefore sees exactly the same mismatch even though the inferred state has changed. The example does not require the audience to accept a complicated security claim on faith; every number can be checked mentally. Explain that the attack succeeds within this illustration because both readings can be altered coherently and there is no trustworthy independent anchor. An additional language model examining the same observations does not create that anchor. A useful agentic workflow must recognize information limits, request appropriate evidence or hold the action. The network version follows the same geometry through the measurement matrix.

**Predict before revealing** Does LLM voting add an independent measurement?

**Explanation** No. New physical evidence must come from outside the corrupted readings.

**Sources.** Original scalar adaptation of the false-data-injection principle in Liu, Ning and Reiter, CCS 2009.

### Frame 93: Weighted least squares fits the measurement model

Translate each symbol into an object the audience already understands. The measurement matrix predicts sensor readings from a candidate state. The estimator chooses the state that minimizes the weighted squared disagreement. The covariance describes uncertainty in each channel; the implementation assumes independent errors, so it is diagonal. A precise channel receives more weight because a small discrepancy there is more surprising. Different measurement types may have different physical units, which is why each residual is scaled by its own standard deviation. In the fresh four-bus exercise there are eight measurements and three identifiable unknown angles. The matrix has full column rank, so the linear weighted fit has a unique state estimate. These facts make the following residual degrees-of-freedom calculation meaningful. They do not establish trustworthy provenance for the measurements or correctness of the network topology.

**Sources.** Classroom_Labs L8; standard_formulations.weighted_least_squares. WLS: Schweppe and Wildes, 1970.

### Frame 94: A residual alarm is a statistical test under stated assumptions

Define the residual before defining the test statistic. Each squared residual is normalized by its variance, producing a dimensionless sum. With a correct full-rank linear model and independent Gaussian errors of the stated covariance, the statistic follows a chi-square distribution with the number of measurements minus the number of estimated states degrees of freedom. Here that is five. A one-percent upper-tail threshold is about fifteen point zero nine. This is a calibration statement under assumptions, not a guarantee in an arbitrary real network. An alarm can reflect model error or ordinary sensor failure. Conversely, a missing alarm cannot establish that data are authentic. Ask participants to recall the two sensors shifted together. The next slide shows why that arithmetic example extends exactly to a family of coordinated changes in the linear network model.

**Predict before revealing** Does “no alarm” mean “safe to control”?

**Explanation** No. It means this test found no excessive model inconsistency. Provenance and operating limits remain separate checks.

**Sources.** Residual test implementation: standard_formulations.weighted_least_squares. State estimation foundations: Schweppe and Wildes, 1970.

### Frame 95: The cancellation is visible in three lines

Reveal the three lines separately and pause before the last cancellation. The vector c is a change in the inferred state, and multiplying it by H produces the corresponding change in all modeled sensor readings. For full-rank linear weighted least squares with the same weights, adding this measurement change shifts the fitted state by c. Subtracting the new predicted readings removes the same added term, leaving the original residual. No mysterious failure of the numerical solver is involved; it solves the altered measurement problem correctly. State the attack assumptions instead of implying that every real system permits this construction. The exercise assumes knowledge of the model and control of all required unprotected readings. Protected independent channels can prevent some shifts from being consistent. The following experiment is confined to a synthetic educational DC model.

**Predict before revealing** Which term disappears when we subtract the predicted measurements?

**Sources.** False-data-injection construction: Liu, Ning and Reiter, CCS 2009. Defensive four-bus exercise in L8.

## Frame 96: Hands-on: sweep isolated and coordinated corruption

Cyber response: consistency and trustworthy evidence

### Hands-on: sweep isolated and coordinated corruption

**Notebook L8** Seven minutes: run the full cell; compare residual and state-error plots.

```
ATTACK_SCALES = (0., .25, .5, 1., 1.5)
started = time.perf_counter()
cyber = lab.cyber_sweep(ATTACK_SCALES)
timings['L8 fresh WLS attack sweep'] = time.perf_counter() - started
display(cyber['table'])
```

**Predict first**

For a growing isolated bias, what happens to $J$? For a growing $Hc$ shift, what happens to $J$ and the state error?

**Read two different quantities**

The residual is observable from measurements and the model. State error uses simulated truth and is an offline evaluation metric.

The scale multiplies a synthetic perturbation. It is not an estimate of attack likelihood or real-world severity.

Exact excerpt: Classroom_Labs L8 lines 1–5. Fixed synthetic seeds and attack directions in src/classroom_labs.py and standard_formulations.py.

**Next:** Which curve can an operator compute without knowing the truth? 96/138

Give participants the full seven minutes to run and inspect the experiment. The attack scale varies over five stated values for both isolated corruption and the coordinated Hc construction. Ask them to predict the two plots before looking at the table. Their explanation should distinguish a detector output from an evaluation metric. The residual statistic can be computed from the reported measurements and the assumed model. The norm of state error requires the true simulated state, which is available to us because this is an experiment. A real operator does not generally know that truth. Do not let the workflow use offline truth as if it were an available verification signal. Participants should record one example where no residual alarm occurs despite a changed inferred state, and explain why additional independent information would be required.

**Notebook excerpt.** Run the complete cell in context.

```
ATTACK_SCALES = (0., .25, .5, 1., 1.5)
started = time.perf_counter()
cyber = lab.cyber_sweep(ATTACK_SCALES)
timings['L8 fresh WLS attack sweep'] = time.perf_counter() - started
display(cyber['table'])
```

**Sources.** Exact excerpt: Classroom_Labs L8 lines 1–5. Fixed synthetic seeds and attack directions in src/classroom_labs.py and standard_formulations.py.

## Frame 97: Same residual does not imply the same state

Read the plots in order. The residual plot shows that a sufficiently large isolated perturbation violates the consistency test, while the coordinated construction retains the original statistic at every sampled scale. The state-error plot reveals that the coordinated corruption still changes the inferred state. Mention the residual-axis scaling shown on the figure so students do not interpret vertical distances as an ordinary linear scale. At scale one, the isolated statistic is about two hundred seventy-seven, well above fifteen, while the coordinated statistic remains about three point nine. Those are reproducible outcomes for the stated synthetic seeds and model, not general attack-detection rates. The right-hand plot is an offline scientific audit using known truth. The next workflow decision must instead rely on information actually available to the estimator and the

declared provenance assumptions.

**Sources.** Original computed figure: Classroom_Labs L8 / classroom_cyber.png. Coordinated $J \approx 3.92022$ remains constant across the sampled scales.

### Frame 98: Awareness must distinguish consistency from trustworthy provenance

This slide places the statistical lesson inside the agentic workflow. Awareness should produce separate fields for residual consistency and measurement provenance. A low residual supports only the first field. If a cyber incident or external evidence makes the measurements suspect, the workflow should ask whether trustworthy replacement channels exist and whether they make the state identifiable. The language model can summarize the missing evidence, request an authorized tool or explain why the action is held. It cannot create a trustworthy measurement by reasoning more fluently. Multiple model responses based on the same corrupted data have the same information limitation. The exercise deliberately includes a HOLD branch when replacement channels are unavailable. This is not a claim that the residual test itself detects the stealth case; that would contradict the counterexample we just demonstrated.

**Predict before revealing** Can an LLM vote resolve missing observability or missing trusted channels?

**Explanation** No. Voting over the same evidence does not add an independent physical constraint.

**Sources.** Information limitation illustrated by Liu et al., 2009; explicit protected-channel assumption in Classroom_Labs L9.

### Frame 99: Recovery assumes three independent protected angle readings

Be explicit that the recovery channels are assumed, not discovered by the LLM or authenticated by this notebook. There are three independent angle readings, one for each unknown angle, with the stated small noise level. The identity measurement matrix has rank three, so the state is identifiable from this trusted set. However, three measurements for three unknowns leave zero residual degrees of freedom. The estimator can fit them exactly, so it has no redundant chi-square test for detecting an error within this set. Trust must come from the external measurement and security assumptions. This distinction prevents the recovery demonstration from becoming a circular claim that a zero residual proves security. Ask participants what additional engineering would be needed to establish that trust in practice. The tutorial then uses the recovered estimate for a small constrained redispatch calculation.

**Sources.** Exact classroom recovery model: standard_formulations.run_wls_demo; Classroom_Labs L9. Protected measurements are assumed trustworthy.

### Frame 100: Corrective control: move a stressed flow back inside its limit

Before showing a quadratic program, make the control action concrete. An injection at node two, balanced by the reference source, changes branch flows through a known DC sensitivity. For the stressed branch, the sensitivity is minus four-sevenths. In this separate noiseless illustration, an estimated flow of zero point two four must be brought to zero point one seven to preserve a zero point zero one margin below the physical zero point one eight limit. Dividing the needed flow reduction by the sensitivity magnitude gives a minimum injection of zero point one two two five. The real classroom solve uses a noisy protected estimate, so its optimum is slightly different. Label that distinction carefully. The diagram does not use simulated truth as an operational gate; it simply teaches how a linear flow sensitivity becomes an action constraint.

**Sources.** Sensitivity $FB^{-1}[0, 1, 0]^T$ from standard_formulations._redispatch. Original noiseless arithmetic illustration; all quantities here are per unit.

### Frame 101: Planning uses a constrained quadratic control problem

Read the objective as choosing the smallest squared corrective injection. The constraints keep all predicted branch-flow magnitudes within their ratings minus the engineering margin, and restrict the injection to its allowed interval. Define every matrix by its role instead of asking students to memorize the notation. The vector e-two selects injection at the second non-reference node, while the reference source balances the change. For this fixed linear model, the resulting scalar problem is convex. The implementation uses SLSQP and independently checks finite values, action bounds and predicted flow constraints after solving. The protected estimate is an input with an explicit trust assumption. The two code lines compare that assumption being available or unavailable; they do not change the numerical control objective. This distinguishes an information gate from a contest between optimization algorithms.

**Notebook excerpt.** Run the complete cell in context.

```
without_trust = lab.trusted_response(cyber, protected_available=False)
with_trust = lab.trusted_response(cyber, protected_available=True)
```

**Sources.** Exact excerpt: Classroom_Labs L9 lines 1–2; constrained model and checks in standard_formulations._redispatch. DC illustration, not AC feeder resilience.

### Frame 102: Hands-on: evidence availability changes the permitted response

Allow five minutes for the full cell and discussion. The no-trust condition returns HOLD rather than an invented replacement state. The protected condition proposes a flexible injection of about zero point one two one six nine per unit. The predicted stressed-branch flow is at the zero point one seven planning target. When the experiment separately evaluates that same action against its known true state, the actual modeled flow is about zero point one seven zero four six, still below the physical limit. These numbers have different roles. Predicted constraints support the candidate under the trusted-estimate assumption. Truth-based flow and violation count are offline performance audits. A successful audit on one seeded example is not a robustness guarantee against arbitrary measurement error or model mismatch. Ask participants to state the protected-channel assumption in their own words before moving on.

**Predict before revealing** Why is the offline value not an input to the operator's verification rule?

**Explanation** The real system does not reveal its exact state just because our simulator knows it.

**Sources.** Fresh L9 output with fixed synthetic seeds; standard_formulations.run_wls_demo. The zero-violation offline audit applies to this modeled case only.

### Frame 103: Every stage has a specific numerical or evidentiary job

Return to the five-stage vocabulary after the audience understands the numerical problem. Awareness computes an estimate and a consistency statistic, but also records rank and trust conditions. Planning solves the constrained redispatch problem only when its required inputs are admissible. Simulation evaluates the action in the stated DC model. Verification checks provenance and modeled limits and can hold the recommendation. The language optimizer may select from permitted refinements and request another simulation; it cannot waive those gates. This case uses a quadratic program because that is the natural form of the stated decision. It would be misleading to label every calculation MPC or DRL for consistency of branding. L8 and L9 are fresh numerical exercises, while L10 supplies the full archived language trace. That distinction keeps live execution, replayed evidence and conceptual stage roles honest.

**Sources.** Classroom_Labs L8–L10; archived cyber_stealth / dc run in proposal_complete_workflow.json. Fresh numerical fixtures and archived LLM calls are separately labeled.

### Frame 104: Actual Llama response: request a new checked candidate

Present these as observable response fields from the saved trace, not as access to hidden internal reasoning. The short planning reply requests candidates. The optimizer reply selects a permitted trusted-QP refinement and explicitly asks for another simulation and verification. The reported model-call times are about eight hundred twenty-nine milliseconds and one point one nine five seconds in the archived environment. The final numerical result comes from the constrained solver and its checks, not from the prose. The response does not prove that the trusted measurements are authentic, that the DC model is adequate for a distribution feeder, or that an operator has authorized execution. Those remain separate conditions. This example shows where a language response can help explain a workflow step while leaving authority with defined tools, evidence and approval. The next slide tests the final outcome against the fixed supervisor.

**Sources.** Exact returned rationale fields: archived cyber_stealth / dc trace, proposal_complete_workflow.json. Model-call wall times; rounded only outside quotes.

### Frame 105: A matched baseline avoids crediting the LLM for better information

The fixed and Llama supervisors reach the same action and objective in this archived DC case. Their total elapsed times differ substantially, with the fixed supervisor completing in a few milliseconds and the language-guided workflow taking about five seconds. These are single-run measurements, not distributions or universal performance claims. The comparison is meaningful because the numerical objective, tools and evidence assumptions are matched. If the baseline had only corrupted measurements while the agent received protected measurements, any benefit would combine better information with orchestration. Avoid making that unfair attribution. The scientific lesson is that protected information and constrained control enable the recovery in this example. The agentic layer can organize requests, reasons, holds and evidence, but its additional value should be evaluated through explicit workflow and human-performance measures, alongside its latency and failure modes.

**Predict before revealing** Would comparing untrusted baseline data with trusted agent data be fair?

**Explanation** No. That would mix the value of additional information with the value of orchestration.

**Sources.** ARCHIVED ACTUAL: fixed versus agentic cyber_stealth / dc, proposal_complete_workflow.json. Same objective and evidence assumptions; one recorded run, four Llama calls.

### Frame 106: The control-room handoff is a decision packet, not a green badge

End the cyber chapter by returning to the operator rather than to an optimizer score. A useful recommendation includes its proposed action, the objective, the measurement trust assumptions, the model used, the relevant revision and the outcome of each required check. The operator can then acknowledge, revise or reject it. After any authorized action, readback should establish what state was actually reached; an acknowledgment alone is not physical confirmation. Across the three cases, the numerical tools differ because the decisions differ. Voltage support uses continuous-control methods, restoration adds discrete topology and pickup choices, and cyber response exposes information and provenance limits before constrained control. The common agentic framework is a disciplined flow of requests, tools, checks and human decisions. The closing section will evaluate that workflow without confusing documentation quality with electrical performance or claiming unmeasured benefits.

**Predict before revealing** What did the three cases teach about choosing numerical methods?

**Explanation** Continuous voltage control, discrete restoration and evidence-limited cyber response need different tools within one auditable workflow.

**Sources.** Case synthesis: Classroom_Labs L1–L10; operator readback exercise L12. The tutorial dashboard is an educational interface, not a deployed control system.

# 6 Evaluation and operator decisions

### Frame 107: What did the framework actually improve?

Ask learners to answer this using one case they just ran. A study can finish while its candidate is infeasible. A rejected candidate can demonstrate a working gate while consuming time. A numerical improvement can come from a stronger optimizer rather than the LLM. Put those observations in different columns. The tutorial establishes an executable learning workflow with visible intermediate decisions; it does not yet establish a benefit in a utility control room. Invite the audience to name the smallest extra experiment needed for their preferred claim. A claim about staff workload requires participants and a workload measure. A claim about electrical feasibility requires a stated model, operating limits and test scenarios. Transition: make the comparison fair before interpreting any score.

**Predict before revealing** Which claim is supported by a low cost and a fluent explanation?

**Sources.** PowerAgentBench-SS (2026), §§II–III; Executed tutorial: Classroom_Labs, L0–L12

### Frame 108: Compare methods on the same task

Use the same demand realization, topology, operating limits, action bounds and final electrical check for a paired comparison. A method that sees more cases or gets extra simulations has received additional information or compute. Record that explicitly. The fixed baseline may use the same MPC, SAC policy and verifier as the agentic workflow; this isolates orchestration rather than changing the entire controller. Report the objective among feasible candidates and also report the failure fraction. An average objective that silently removes failed cases can favor a brittle method. In the held-out response pilot, all pool selectors paid for all three LLM calls. The learned selector therefore cannot claim the latency of only the chosen response. Transition: choose the ablation that answers the claim.

**Sources.** PowerAgentBench-SS (2026), §§II–III; Recorded voltage pilot: E1–E8

### Frame 109: Ablate one mechanism at a time

Build the comparison from left to right. First establish a deterministic workflow that solves the engineering problem. Add language interpretation while preserving its tool contract. Then introduce specialist handoffs and measure whether they reduce malformed requests or unsupported reports on varied tasks. Finally compare selection rules under the same candidate pool and fresh-trial budget. This is a recommended experimental design, not a claim that every ablation was run in this package. The implemented pilot directly compares several response selectors and fixed numerical refinement; the real LangGraph exercise separately checks orchestration equivalence against direct Python. Keep those scopes visible. Ask which variant would isolate the value of retrieval or a checkpoint. Transition: a recent benchmark formalizes this separation between tools and scoring.

**Sources.** PowerAgentBench-SS (2026), §§II–III; Executed LangGraph teaching lab: G0–G5; Recorded voltage pilot: E1–E8

### Frame 110: Frontier: evidence-backed evaluation

PowerAgentBench-SS separates an agent-visible tool environment from hidden evaluation. Its instantiated pilot studies DC thermal contingency analysis, so it is useful here as an evaluation pattern rather than evidence for our voltage controller. For the classroom, draw a boundary around what the agent may observe and place the final scoring calculation outside it. Training feedback and

selector features must not contain the held-out answer. A correct statement can still lack supporting evidence in the submitted trace, which is why claim support deserves its own measure. Have a learner point to the tool record supporting one numerical sentence in an operator report. If the record is absent, mark the sentence unsupported even if its value happens to be true. Transition: operating uncertainty is another dimension the workflow must expose.

**Sources.** PowerAgentBench-SS (2026), §§II–III

### Frame 111: Frontier: request uncertainty studies

CONDUCTOR provides a recent example of language orchestration around numerical grid-analysis engines, including uncertainty-aware analysis. Its reported measurement replay must not be described as a forecast-performance experiment. Relate that distinction to our load-scale exercise: changing a scenario is a sensitivity study, while a probability claim needs a justified distribution and calibration. A useful extension is to provide a small declared scenario set, run the same candidate through every scenario, and summarize the observed margins. That extension is proposed here; it is not an experiment already contained in the saved classroom output. Ask what additional data would justify the chosen scenarios for a real feeder. Transition: uncertainty about the grid and authority to act are separate concerns.

**Sources.** CONDUCTOR (2026), uncertainty-aware orchestration; Executed tutorial: Classroom_Labs, L0–L12

### Frame 112: Frontier: enforce authority in software

The governance-aware TSO digital-twin paper uses an external control layer for tool permissions, budgets and approval. Its reported prototype distinguishes task success from compliance with those enforced rules. Bring that distinction into our notebook: a candidate can satisfy the advisory readback rule without becoming a field command. Conversely, a plausible language answer cannot make an unavailable permission appear. Draw the authorized path through the review gate and show the attempted bypass ending at HOLD. This is an architectural lesson, not a claim of production certification. Ask learners to identify which checks run outside the language model in L12. The answer includes candidate identity, evidence binding and required model scope. Transition: now test what the existing experiments actually support.

**Sources.** Governance-aware TSO digital twin (2026); Executed tutorial: Classroom_Labs, L0–L12

### Frame 113: L11: one optional local Llama call

Run the complete L11 cell after the earlier engineering cells. Its default branch prints that no new model call occurred. The optional path requires RUN_LOCAL_LLAMA=True in L0 and a prepared local Ollama service containing llama3.1:8b. It asks for a choice from the bounded catalog on the original test_01 context, then writes the actual response to a new journal. Do not silently apply that response to a modified classroom case. The quoted public reason comes from the separate 17 October validation and is a saved actual output, not a new live response generated by these slides. It proposes reducing reactive effort and uses cautious language about possible cost improvement. A fresh numerical trial is still required. Transition: inspect where the measured call spent its time.

**Notebook excerpt.** Run the complete cell in context.

```
client = ProposalLlama(journal, model='llama3.1:8b')
live_reply = client.ask('Optimization Agent', pool['context']['public_context'],
                        list(pool['context']['catalog']))
```

**Sources.** Ollama chat API; Classroom L11 and saved actual validation

### Frame 114: Where did one Llama call spend its time?

This is the saved actual Llama 3.1 8B call from the optional classroom validation, not a new call made by the slide. Ollama reports its loading, prompt-processing and generation durations; the client timer includes the complete request. Small differences include server and transport overhead, so do not force the components to equal the client total exactly. The response chose RELAX_Q_10_PERCENT on the original test_01 context and requested a newly checked trial; it did not certify that future trial. Warm loading, prompt length and hardware can change latency substantially. Ask the audience whether this call belongs in a subsecond control loop or an advisory study. A deadline claim would require repeated end-to-end measurements, including failures and fallback. Transition: present this evidence in an operator-facing packet.

**Sources.** Ollama chat API: timing fields; Executed tutorial: Classroom_Labs, L0–L12

### Frame 115: How the workflow could fit beside control-room tools

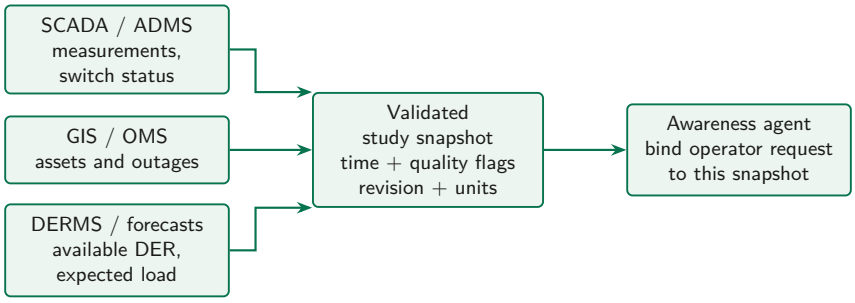
Follow the data path from the existing operational applications to a read-only study context, then through numerical candidates and verification to a reviewable recommendation. This is a deployment concept. The notebooks use a synthetic feeder and teaching cases; they do not connect to a live control system. The diagram should make the human review point prominent without suggesting that a dashboard alone establishes useful human oversight. An operator needs the relevant evidence at the right time and must be able to reject or request clarification. Ask which information belongs in a structured field rather than generated prose: equipment identifiers, limits, model scope and candidate identity are strong examples. Transition: trace the operational inputs, then rehearse each case before inspecting the review packet.

**Sources.** Governance-aware TSO digital twin (2026); Executed tutorial: Classroom_Labs, L0–L12

### Frame 116: The study starts with a consistent operating snapshot

Evaluation and operator decisions

**The study starts with a consistent operating snapshot**




SCADA: supervisory control and data acquisition; ADMS: advanced distribution management system. GIS: geographic information system; OMS: outage management system; DERMS: distributed energy resource management system.

**Predict:** What if the switch status and model describe different network revisions?

**Explanation:** Hold the study and reconcile the snapshot before calculating a recommendation.

Proposed integration, not implemented live. NLR ADMS Test Bed; tutorial L1 and L7 demonstrate input/evidence binding.

**Next:** How do the notebook functions become services? 116/138

Begin with the systems an operator already uses. SCADA and an ADMS provide operational measurements and equipment status. A geographic information system supplies asset and connectivity information, while an outage management system supplies outage context. A DER

management system can provide controllable resource availability. The diagram is a proposed interface pattern, not a claim that these systems are connected to the tutorial. Each utility has its own system boundaries and authoritative records. The first engineering task is to reconcile identifiers, units, timestamps, quality flags and topology into one study snapshot. A language model cannot resolve contradictory physical status by choosing a plausible sentence. L1 and L7 illustrate the narrower input-binding mechanism. For a live deployment, authentication, source integrity, clock handling and model maintenance are additional work. Ask the audience which system is authoritative for switch status in their organization.

**Predict before revealing** What if the switch status and model describe different network revisions?

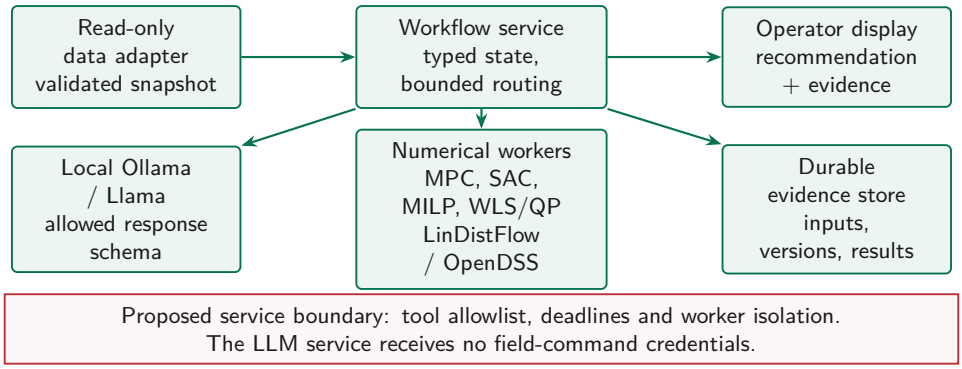
**Explanation** Hold the study and reconcile the snapshot before calculating a recommendation.

**Sources.** Proposed integration, not implemented live. NLR ADMS Test Bed; tutorial L1 and L7 demonstrate input/evidence binding.

## Frame 117: The assistant runs beside the existing control system

Evaluation and operator decisions

### The assistant runs beside the existing control system




Proposed architecture. NIST SP 800-82r3, §5.2.3.1; LangGraph persistence; tutorial G0–G5 and L0–L12.

**Next:** What appears on screen during a voltage alert? 117/138

Map each box back to something the audience has already run. The numerical functions become bounded jobs that accept typed inputs and return results, scope and timing. The workflow service coordinates these jobs. A local Llama service interprets requests or proposes allowed refinements, with responses validated against the allowed schema. Running Ollama locally does not by itself create an operational security boundary. A deployment needs controlled network paths, identities, restricted tool permissions and a durable evidence store. The graph lab's in-memory checkpoint is useful for teaching but does not survive a process restart; persistence and recovery must be engineered and tested. The tutorial's engineering supervisor and its separate LangGraph lab are also distinct implementations. Integrating them into this service architecture is proposed development. There is no implemented ADMS connector or equipment command gateway in this tutorial.

**Sources.** Proposed architecture. NIST SP 800-82r3, §5.2.3.1; LangGraph persistence; tutorial G0–G5 and L0–L12.

## Frame 118: Voltage desk: an alarm becomes a request for a checked action

This screen is a teaching storyboard, not a screenshot of a deployed utility product. Start with the operator's request and underline full feeder. The model scope is part of the request, not a footnote added after solving. MPC and the saved SAC policy generate alternatives, and simulations evaluate their electrical consequences. The twelve-case result comes from the recorded model audit and must not be described as twelve events on a live network. Its value here is to show why a narrow green check can be misleading. The framework can organize the evidence, propose a permitted next study and make the missing scope visible. It has not demonstrated a reduction in operator workload; that requires a human comparison. A fixed supervisory application with the same scope check can also hold this request. The potential language benefit is interpreting varied requests and explaining tool evidence accurately.

**Predict before revealing** What would a useful assistant save the operator from doing manually?

**Explanation** Assembling model scope and conflicting checks into one reviewable record.

**Sources.** Illustrative scenario. 12-case audit: recorded paired-model pilot. One-case demo: L5; scope packet: L12. No field dispatch occurred.

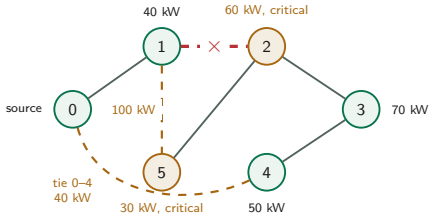
## Frame 119: Restoration desk: the service priority is visible before review

Evaluation and operator decisions

### Restoration desk: the service priority is visible before review

**Illustrative operator request**

"The fault is isolated. Compare pickup plans that prioritize the two critical loads."




Six-node fixture; isolation is assumed in this story.

| Plan | Total kW | Critical kW |
|---|---|---|
| Unweighted | 140 | 30 |
| Weighted | 130 | 90 |

**Proposed operator display**

Requested policy: critical-load weights.
Candidate: weighted MILP solution.
Next: engineering and switching review.

Awareness: status. Planning: MILP. Simulation: candidate network. Verification: limits and evidence. LLM Optimizer: request a permitted refinement.

**Predict:** Does the higher critical service prove an LLM advantage?

**Explanation:** No. It follows from the objective. A fixed workflow with the same MILP can achieve it.

Recorded Classroom_Labs L6–L7. Active-power transport model, not an AC switching certification. NLR ADMS Test Bed illustrates a route to lab integration.

**Next:** Where does Llama help interpret the operator's words? 119/138

Ask a learner to point to the failed segment and the two ties before discussing the table. This reconnects the control-room request to the physical topology taught earlier. The statement that the fault is isolated is an assumption in this illustrative story; the notebook does not validate work permits, field isolation or protection coordination. Both numerical rows are real outputs from the classroom fixture. Their difference reflects different service objectives. The assistant's proposed contribution is to interpret and confirm the requested priority, call the appropriate study, preserve the chosen objective and summarize the tradeoff. It does not create the mathematical gain. The operator still needs the operational switching procedure and the utility's engineering checks. Explain that the classroom transport calculation is a limited simulation; a utility pilot would add suitable AC, protection and temporal switching studies before treating the recommendation as operationally actionable.

**Predict before revealing** Does the higher critical service prove an LLM advantage?

**Explanation** No. It follows from the objective. A fixed workflow with the same MILP can achieve it.

**Sources.** Recorded Classroom_Labs L6–L7. Active-power transport model, not an AC switching certification. NLR ADMS Test Bed illustrates a route to lab integration.

## Frame 120: A useful Llama role: understand the operator's restriction

Evaluation and operator decisions

### A useful Llama role: understand the operator's restriction

**Saved operator request h06**

"Use the 0.25 km bypass to reconnect demand, with current no higher than **100 A**. Keep the 2 km bypass **open** and leave PV generation at **zero**."

The request restricts a permitted study; it cannot remove fault quarantine.

interpret

**Actual saved Llama fields**

```
{
  "status": "READY",
  "rating_fraction": 1.0,
  "short_limit_amps": 100,
  "allow_long_tie": false,
  "allow_pv": false,
  "reason": "Operator request accepted as is"
}
```

`READY` is an interpretation status; numerical feasibility still requires a separate check.

| Seven authored requests (post-hoc subset) | Documented parser | Parser + Llama |
|---|---|---|
| Clear requests completed correctly | 3/5 | 5/5 |
| Missing/prohibited requests clarified | 2/2 | 2/2 |
| Actual Llama calls across this subset | 0 | 4 |

**Saved h06 call: 3.160 s.** Historical feeder catalog + OpenDSS, separate from L6's six-node MILP.
The observed gain is request interpretation against this parser; better optimization and operator-time savings were not established.

Actual saved h06 request, schema, response and client duration; Recorded restoration comparison: five clear and two clarification cases.

**Next:** When measurements are suspect, what evidence can the operator trust? 120/138

Begin with the operator's request and ask the audience to identify its three restrictions before revealing the response. The short bypass has a 100 ampere ceiling, the long bypass is unavailable by instruction, and PV must remain zero. Llama returns those restrictions in a fixed schema; rating_fraction stays at the trusted default of one. READY describes interpretation only. The recorded h06 call took 3.160 seconds of client elapsed time. Across this descriptive subset, the documented parser completed three of five clear requests, while parser followed by Llama completed all five. Both clarified both missing or prohibited requests. Four Llama calls were used in total. These historical feeder catalog experiments are separate from L6's six-node MILP. The evidence supports request interpretation against this parser, not improved optimization or measured operator workload. Next, contrast a clear operator request with measurements whose trust is uncertain.

**Sources.** Actual saved h06 request, schema, response and client duration; Recorded restoration comparison: five clear and two clarification cases.

## Frame 121: Cyber desk: a quiet residual still needs a trustworthy basis

The residual value illustrates the coordinated corruption case, where the measurements can remain internally consistent while the inferred state is wrong. It is a recorded synthetic result, not an attack detector for a live utility. The trusted-data branch uses protected angle measurements that the teaching code assumes are available and trustworthy. A deployed system must establish that basis through operational security and measurement provenance; an LLM cannot supply it by assertion. The constrained quadratic program produces the reported action, and the synthetic true flow is available for offline evaluation. A control room would not have unrestricted access to hidden ground truth. The fixed and language-assisted supervisors return the same correction in this trace. The

potential workflow value is explicit escalation, tool coordination and explanation, while the correction itself comes from the numerical method.

**Predict before revealing** Can an LLM explanation establish that a sensor is trustworthy?

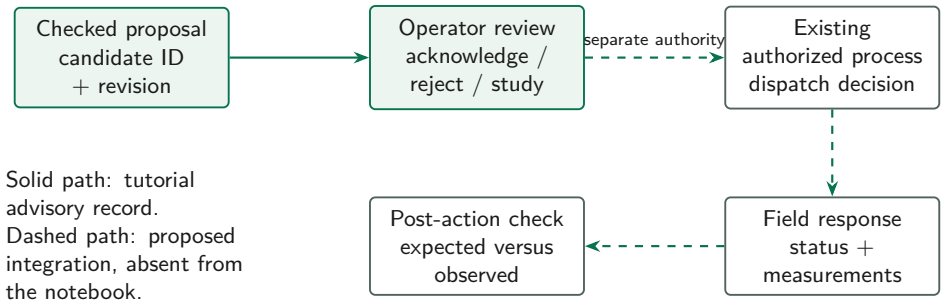
**Explanation** No. Provenance and independent operational evidence must support that assumption.

**Sources.** Recorded Classroom_Labs L8–L9 and archived trace. Liu, Ning & Reiter, CCS 2009; protected angles are assumed trusted in this teaching model.

## Frame 122: Operator acknowledgment and field execution are different events

Evaluation and operator decisions

### Operator acknowledgment and field execution are different events




**Predict:** If the device does not reach the expected state, should the assistant mark the job complete?

**Explanation:** No. Preserve the discrepancy and hand it to the established operating procedure.

Proposed integration and tutorial L12 boundary. NIST AI RMF: GOVERN 3.2, MANAGE 2.4 and 4.1.

**Next:** How should delayed or unavailable advice be handled? 122/138

Use the two rows to distinguish two kinds of readback. In the tutorial, the human acknowledgment names a checked candidate. In an operational deployment, field readback means observing equipment state and measurements after an authorized command. These are separate events and need separate records. The notebook implements only advisory acknowledgment, and its equipment-dispatch flag remains false. A future command gateway would require independent authorization, equipment interlocks and the utility's operating procedure. The proposed post-action check compares expected and observed consequences and preserves discrepancies instead of automatically declaring success. It must not issue an improvised rollback, because the safe response depends on the current physical state. Ask participants to name the system and role responsible for dispatch in their own control room. The assistant supports that responsibility rather than assuming it.

**Predict before revealing** If the device does not reach the expected state, should the assistant mark the job complete?

**Explanation** No. Preserve the discrepancy and hand it to the established operating procedure.

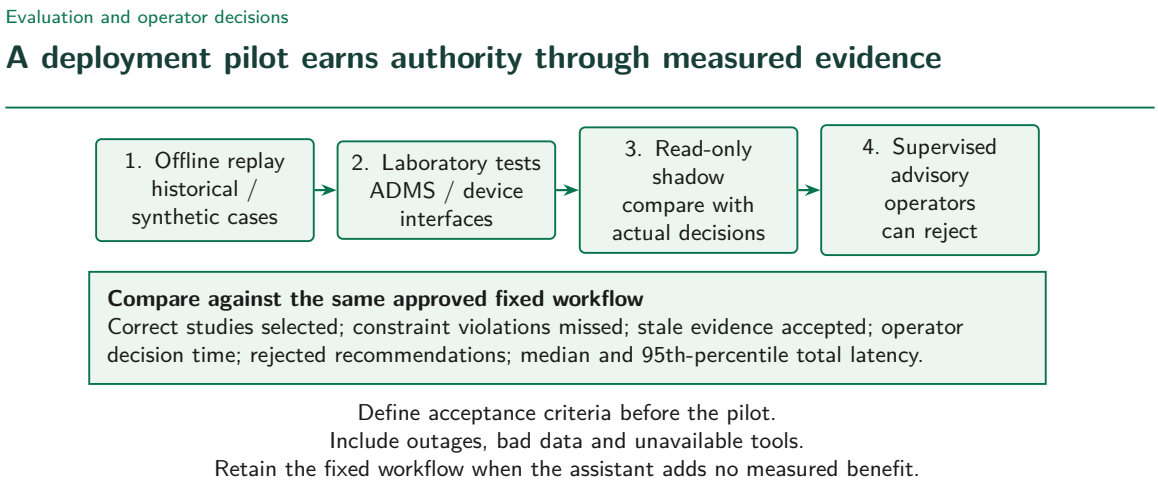
**Sources.** Proposed integration and tutorial L12 boundary. NIST AI RMF: GOVERN 3.2, MANAGE 2.4 and 4.1.

### Frame 123: Slow advice must not delay the existing control response

The observed Llama duration is useful because it prevents treating a language-model call as an instantaneous part of a protective control loop. It is a single recorded call, including model loading, so it does not establish a latency distribution or an operational service-level target. A pilot should measure warm and cold calls, queueing, tool execution and total recommendation time. The proposed validity check makes freshness explicit. Its deadline must come from the use case and the utility's operating requirements, not from an arbitrary number on a slide. A changed topology or expired snapshot can invalidate an otherwise well-formed result. The teaching notebook demonstrates revision binding but does not implement a production deadline scheduler. On advisory failure, established protection and approved control procedures continue; the assistant reports the incomplete study without inventing a replacement control action.

**Sources.** Recorded L11 call; Ollama timing fields. Proposed expiry/failure handling informed by NIST SP 800-82r3, OT performance and availability.

### Frame 124: A deployment pilot earns authority through measured evidence




Proposed staged pilot, not a mandated sequence. NLR ADMS Test Bed; NIST AI RMF MEASURE 2.3; DOE Grid Control and Data Science.

**Next:** Which parts can we rehearse with the current notebook? 124/138

Describe this as a proposed evaluation path. The cited laboratory demonstrates that realistic ADMS and DERMS integration can be tested with simulated networks and hardware interfaces; it does not validate our agentic workflow. Offline replay first checks reproducibility and failure behavior. Laboratory integration then exposes interface and timing issues. Read-only shadow operation consumes approved data and records recommendations while the established workflow remains authoritative. A supervised advisory pilot adds operator interaction after acceptance criteria are met. There is no automatic graduation to autonomous control on this slide. For scientific evaluation, match the cases, objectives, available information and computational budgets. Measure human decision quality and time with an appropriate study design before claiming workload savings. Report how often the baseline is better, and include failures and latency tails rather than only successful average cases.

**Sources.** Proposed staged pilot, not a mandated sequence. NLR ADMS Test Bed; NIST AI RMF MEASURE 2.3; DOE Grid Control and Data Science.

## Frame 125: A four-minute rehearsal connects the desk to the notebook

Allow four minutes within the operator segment. This activity uses already computed outputs rather than requiring a new model installation or a long Llama call. Assign one event to each pair so they have time to inspect evidence instead of skimming all three cases. The person playing the assistant should point to a field or number, identify its model scope and explain the next step. The operator should ask one challenge question, such as whether the topology changed, whether the full feeder was checked or why the trusted measurement assumption is justified. Swap roles if time permits. This rehearsal is a teaching activity, not a measured usability experiment. Its purpose is to connect the mathematical and software lessons to a recognizable control-room exchange. The next slides run the exact L12 acknowledgment checks.

**Sources.** Tutorial Classroom_Labs L5–L9 and L12. Rehearsal is educational; the saved outputs do not certify a real operating decision.

## Frame 126: The operator sees scope before readback

Evaluation and operator decisions

### The operator sees scope before readback

**Saved recommendation ID:** `b67cff9c8eb444c2`

| Packet field | Full MV/LV request | Explicit MV teaching request |
|---|---|---|
| Verified scope | MV phases only | MV phases only |
| Scope matches | No | Yes |
| State | HOLD | READY_FOR_READBACK |
| Acknowledged | False | True |
| Equipment dispatch | False | False |

**Predict:** The numerical evidence is the same. Why is only the second packet eligible?

**Explanation:** The second request asks only for the scope that was checked. The first asks for more. Acknowledgment still does not dispatch equipment.

Executed tutorial: Classroom_Labs, L0–L12; Governance-aware TSO digital twin (2026)

**Next:** Acknowledgment is a guarded transition 126/138

Present the action and its identity first, then the supporting evidence. A reviewer should not have to reconstruct the proposal from a long chat transcript. Include the requested and verified model scope, the age of the evidence, numerical constraint margins, rejected alternatives and a concise public rationale tied to tool results. In this tutorial the most consequential scope distinction is the MV-only result versus the broader MV/LV requirement. A polished summary cannot silently expand that scope. Invite learners to remove one field and predict the resulting ambiguity. Removing the identifier obscures which candidate was reviewed; removing the scope can turn a valid limited statement into an unsupported general claim. Transition: acknowledgment changes a specific state, not the physical network.

**Predict before revealing** The numerical evidence is the same. Why is only the second packet eligible?

**Explanation** The second request asks only for the scope that was checked. The first asks for more. Acknowledgment still does not dispatch equipment.

**Sources.** Executed tutorial: Classroom_Labs, L0–L12; Governance-aware TSO digital twin (2026)

### Frame 127: Acknowledgment is a guarded transition

Animate the review path one state at a time. The conceptual sequence distinguishes calculation, review readiness and human acknowledgment. In the implementation, a full MV/LV request supported only by an MV result is held; an explicitly limited MV teaching request can become eligible for acknowledgment. The candidate identifier binds the readback to a particular packet. Emphasize that these are advisory application states. They neither authenticate a real operator nor dispatch equipment. The earlier LangGraph G5 exercise demonstrated interruption and resumption using a checkpoint, while L12 demonstrates the engineering packet checks. They teach related ideas through separate code paths. Ask the audience to identify which failure should stop the path before any readback is accepted. Transition: inspect the exact L12 statements.

**Sources.** Executed tutorial: Classroom_Labs, L0–L12; Executed LangGraph teaching lab: G0–G5; LangGraph: interrupts and resume

### Frame 128: Code: bind review to the candidate

Read these two notebook lines aloud. The first call deliberately supplies the wrong recommendation identifier. The second uses the identifier carried by the eligible MV teaching packet. Both calls go to ordinary Python logic, so the readback rule is inspectable independently of language-model output. The string Student operator is an instructional placeholder rather than an authentication mechanism. Keep the previously constructed packet visible: it records what was checked and under which scope. This exact excerpt comes from L12 of Classroom_Labs, and learners should run L0–L5 first so the selected-trial context exists. Ask them to predict the two outcomes before running the cell. Transition: run the full packet exercise, then inspect its assertions.

**Notebook excerpt.** Run the complete cell in context.

```
wrong_id = lab.acknowledge(mv_scope, 'wrong-id', 'Student operator')
readback = lab.acknowledge(mv_scope, mv_scope['recommendation_id'], 'Student operator')
```

**Sources.** Executed tutorial: Classroom_Labs, L0–L12

### Frame 129: Hands-on: read back, reject, inspect

Give learners four minutes and keep this slide visible. First run the unmodified L12 cell after the earlier classroom cells. Inspect the displayed packet for the full MV/LV requirement and compare it with the explicitly MV teaching packet. Then inspect wrong_id in a scratch cell; it is intentionally not the accepted readback. Finally check the three assertions at the end of L12. Have partners explain the difference between an eligible packet and an acknowledged packet. Avoid changing the original assertion expectations during the first run. For an extension, discuss which fields would change if the requested control changed and which evidence would need to be recomputed. This is a local advisory exercise; the notebook has no equipment-dispatch endpoint. Transition: compare the predicted outcomes with the executed checks.

**Predict before revealing** Which of the three outcomes is permission to dispatch equipment?

**Explanation** None. Even the acknowledged packet is explicitly advisory.

**Sources.** Executed tutorial: Classroom_Labs, L0–L12

### Frame 130: The checks distinguish three outcomes

Show the assertions after learners have made their predictions. They are deliberately more precise than an all-purpose PASS label. The full-scope packet is held because the supporting model is

narrower than the request. The wrong identifier is rejected because it does not name the current packet. The eligible MV packet can be acknowledged, yet its equipment-dispatch flag remains false. These outcomes demonstrate the logic of the teaching application, not authentication, field authorization or human comprehension. Ask a participant to explain why the first failure cannot be repaired by entering the right identifier. It requires suitable evidence for the requested scope or an explicitly changed request, followed by the corresponding review. Transition: examine what happens when the underlying request changes.

**Notebook excerpt.** Run the complete cell in context.

```
assert full_scope['state'] == 'HOLD' and not full_scope['acknowledged']
assert wrong_id['acknowledgment_result'] == 'REJECTED'
assert readback['acknowledged'] and not readback['equipment_dispatch']
```

**Sources.** Executed tutorial: Classroom_Labs, L0–L12

### Frame 131: A smaller current limit changes the decision

Connect this question to L1 and L7, where the classroom explicitly checks stale bindings after a request or candidate changes. The binding helps detect a mismatch between saved evidence and its stated inputs. It does not prove that a sensor is trustworthy, that a model is correct or that an operator is authorized. Ask learners to identify the smallest changed field that should invalidate the old check: a current limit, load scale, topology revision or control setting all provide concrete examples. Their answer should include the affected numerical step, not only a request for a new language explanation. The important habit is to preserve evidence provenance through revision. Transition: apply this habit to three operator decision cards.

**Predict before revealing** Can the old acknowledgment survive a changed control?

**Sources.** Executed tutorial: Classroom_Labs, L0–L12

### Frame 132: Capstone: approve review, hold or request a study?

Give each group one card and ask for a decision plus a concrete next step. Card A requires repair or another study at the requested model scope; a confident Llama sentence cannot override the AC failure. Card B exposes a missing operational prerequisite: a feasible mathematical switching plan is not proof of authorized isolation. Card C exposes incomplete information: an unalarmed residual alone cannot establish trustworthy state estimation under the constructed stealth attack. In each response, separate the numerical tool that can help from the authority or external evidence that the tool cannot invent. Do not assume that retaining an old setting is always safe after a changed topology or measurement regime. Use the final minute to compare the three different gaps. Transition: summarize how all five stages contribute.

**Predict before revealing** For your card, identify one missing fact and the next tool or human step.

**Explanation** A: broader-model repair. B: authorized isolation evidence. C: justified independent measurements or escalation.

**Sources.** Executed tutorial: Classroom_Labs, L0–L12; Liu, Ning & Reiter, false-data injection, CCS 2009

## Frame 133: Five stages, three cases, one evidence discipline

Evaluation and operator decisions

### Five stages, three cases, one evidence discipline

| Stage | Voltage | Restoration | Cyber response |
|---|---|---|---|
| Awareness | Operating point and limits | Fault, ties, load priorities | Measurements and trust assumptions |
| Planning | MPC; SAC candidate | Weighted pickup MILP | Constrained recovery QP |
| Simulation | Network response | Flow on selected paths | Estimated / evaluated flows |
| Verification | Voltage/current and scope | Capacity, connectivity, binding | Residual and protected-data basis |
| LLM Optimizer | Propose a permitted refinement | Explain or request a revised trial | Explain or request additional evidence |

The numerical method changes with the decision. The record passed between stages keeps the action, assumptions and results together.

**Predict:** Which stage would catch a stale network revision?

**Explanation:** The evidence-binding check in Verification; a fluent summary cannot repair the mismatch.

Executed tutorial: Classroom_Labs, L0–L12; Recorded voltage pilot: E1–E8

**Next:** Choose the simplest sufficient workflow 133/138

Invite one learner per use case to trace a single candidate through the five proposed stages. Awareness establishes measurements, constraints and model scope. Planning calls the relevant numerical formulation. Simulation computes the consequence of candidate controls; verification checks the declared requirements and binds the result. The optimizer can request a bounded refinement or retain an incumbent, after which the relevant evidence is checked again. Human review is the subsequent authority boundary, not a sixth language agent. For voltage this includes MPC and SAC candidate comparison; restoration uses a weighted service formulation; cyber response distinguishes estimation evidence from protected-information assumptions. The methods change while the handoff discipline remains. Transition: choose how much orchestration the task actually requires.

**Predict before revealing** Which stage would catch a stale network revision?

**Explanation** The evidence-binding check in Verification; a fluent summary cannot repair the mismatch.

**Sources.** Executed tutorial: Classroom_Labs, L0–L12; Recorded voltage pilot: E1–E8

## Frame 134: Choose the simplest sufficient workflow

Treat this as a design decision rather than a contest between labels. A stable calculation with known inputs can be well served by a fixed workflow. Language interpretation becomes useful when users express varied requests, but the resulting tool calls still need validation. Specialist coordination can help when responsibilities and evidence streams differ; it also adds communication overhead and potential handoff failures. Recent agent-scaling research motivates testing that tradeoff instead of assuming that a larger team is better. The classroom direct-Python and LangGraph comparison is deliberately equivalent on its toy cases: orchestration structure alone should not change the mathematics. Ask participants which part of their own workflow is currently variable or error-prone. Transition: preserve enough information to make that improvement measurable.

**Sources.** Kim et al., scaling agent systems (2026); Executed LangGraph teaching lab: G0–G5

### Frame 135: A reproducible result carries its context

Use this checklist to prepare a reproducible scientific report, not a new permission process. The code cells already distinguish fresh numerical calculations, inference from saved SAC weights and archived actual LLM responses. The optional local Llama call is disabled by default, and its saved validation record includes a timestamp and timing fields. A screenshot of an answer is weaker than the structured record that identifies its request, model and tool results. Include failures and rejected alternatives where they affect interpretation. For the selector pilot, preserve the training, validation and held-out split and state that test-time learning was frozen. Ask learners to locate one such field in the supplied outputs. Transition: turn the remaining limitations into a clear next experiment.

**Sources.** Executed tutorial: Classroom_Labs, L0–L12; Recorded voltage pilot: E1–E8; Ollama chat API: timing fields

### Frame 136: Design your next experiment

Ask every participant to write one sentence of the form: adding this mechanism improves this measurable outcome under these conditions. Good candidates include fewer stale-evidence reports, more correct tool selection for varied language requests, or a better tradeoff between objective and computation under a fixed trial budget. A weak hypothesis simply says that agentic AI is better. Require a baseline that can plausibly win and a condition under which the proposed benefit would not be supported. Suggest a held-out scenario split rather than retuning after viewing every result. If the claim concerns operator workload, the next experiment needs a human study rather than another simulation plot. Transition: use the exit ticket to show what you can explain and run now.

**Predict before revealing** Write one testable benefit and one result that would refute it.

**Explanation** Example: fewer unsupported reports at equal tool budget; equal or higher unsupported-report rate refutes the claimed benefit.

**Sources.** PowerAgentBench-SS (2026), §§II–III; Kim et al., scaling agent systems (2026)

### Frame 137: Exit ticket: explain, run, interpret

Allow three minutes for an individual response followed by a brief exchange with a neighbor. Each learner should cite one notebook checkpoint and one actual observed result. For the graph question, a DAG has no directed cycle, while the implemented repair loop is a bounded cyclic state graph. MPC uses an explicit model during planning; the trained SAC policy supplies a continuous candidate from its learned mapping, and that action is still checked in a simulator. A failed candidate is neither proof that the entire framework failed nor proof of safety by itself; explain what the gate prevented and what remained unresolved. Finally identify why acknowledgment in the teaching dashboard does not dispatch equipment. Transition: return to the operator question that opened the tutorial.

**Predict before revealing** Which checkpoint would you show a colleague to explain the framework?

**Explanation** Choose a checkpoint and support the explanation with its exact inputs, output and stated limitation.

**Sources.** Executed LangGraph teaching lab: G0–G5; Executed tutorial: Classroom_Labs, L0–L12

### Frame 138: From request to a reviewable recommendation

Close by asking the audience to describe the entire process in one sentence without using the word intelligent. A suitable description names the request, the numerical computation, the evidence checks and the operator review. The three cases have demonstrated different engineering formulations inside a shared orchestration pattern. They have also shown where a fixed method ties or beats a learned selector, where a reduced model misses full-feeder violations, and where a quiet residual is insufficient evidence. Those limits make the tutorial scientifically useful. Direct learners to the integrated handout for the symbol definitions, exact code references, exercise answers and citations, and to the native LaTeX source and lecture PDF for the progressive teaching sequence. The next step is a focused experiment on a clearly defined workflow need.

**Sources.** Executed tutorial: Classroom_Labs, L0–L12; Executed LangGraph teaching lab: G0–G5; Recorded voltage pilot: E1–E8